

Université Mundiapolis

Ecole d'Ingénieurs
Cycle Ingénieur : « *Génie Informatique* »
2A CI INF GA

Module : « *Ingénierie des infrastructures Cloud* »
Année universitaire 2025/2026

Rapport

**Projet : Mise en œuvre d'une
infrastructure cloud de supervision
centralisée sous AWS :**

Déploiement de Zabbix conteneurisé pour le
monitoring d'un parc hybride (Linux &
Windows)

Préparé par :

ELYACHIOUI Douaa

Encadré par :

Prof. Azeddine KHIAT

Table des matières

Table des matières	2
Remerciements	4
Liste des sigles et des acronymes	5
Introduction générale	6
Partie 1.....	7
Présentation de l'environnement AWS (Architecture réseau sous AWS).....	7
Présentation de l'environnement AWS (Architecture réseau)	8
Accès au Learner Lab AWS.....	8
Choix de la région AWS	10
Création du VPC	11
Création et configuration de la Passerelle Internet (IGW)	13
Création d'un Sous-Réseau Public	16
Configuration de la Table de Routage.....	19
Création et Configuration des Groupes de Sécurité	20
Partie 2.....	26
Création des instances EC2 (Zabbix, Linux, Windows)	26
Lancement des Instances EC2.....	27
Étape 1 : Accéder au service EC2	27
Étape 2 : Lancer le serveur Zabbix.....	27
Étape 3 : Lancement de l'instance Linux	31
Étape 4 : Lancement de l'instance Windows	32
Partie 3.....	34
Déploiement du Serveur Zabbix	34
Étape 1 : Connexion au serveur Zabbix	35
Étape 2 :	35
Étape 3 : Installer Docker-Compose	38
Étape 4 : Lancer Zabbix avec Docker	39
Étape 5 : Accéder à l'interface Web et se connecter à Zabbix	41
Partie 4.....	43
Configuration des Clients (Agents) et Monitoring et Tableaux de Bord.....	43
Création des agents :	44

Client Linux.....	44
Client Windows.....	46
Monitoring et Tableaux de Bord	49
Visualisation des métriques.....	49
Listes des figures	51
Conclusion.....	53

Remerciements

Je voudrais profiter de ces quelques lignes et de cette occasion qui m'est donnée pour exprimer ma profonde gratitude à **Monsieur Azeddine KHIAT** pour :

- Son **encadrement attentif** et son **engagement constant** tout au long du semestre.
- La **qualité de son accompagnement pédagogique**, qui m'a permis de progresser efficacement dans mes compétences techniques.
- Le soutien et les conseils apportés lors des **travaux pratiques**, qui m'ont permis de maîtriser :
 - Le **cloud computing**,
 - La **conteneurisation avec Docker**,
 - La **supervision et la gestion des systèmes**.
- L'accès à des **formations professionnelles sur AWS et Huawei**, grâce auxquelles j'ai obtenu **deux certifications** renforçant mon parcours académique et professionnel.

Le projet réalisé dans ce cadre a été particulièrement **enrichissant et formateur**, me permettant de :

- Mettre en pratique les connaissances acquises durant les TP.
- Approfondir ma compréhension des **environnements cloud et des infrastructures modernes**.
- Développer des compétences concrètes et applicables dans le domaine de la supervision et de la gestion d'infrastructures.

Je remercie sincèrement **Monsieur Azeddine KHIAT** pour la **qualité de son enseignement**, sa **disponibilité**, et la **valeur ajoutée** qu'il a apportée à ma formation.

Liste des sigles et des acronymes

AWS : Amazon Web Services

EC2 : Elastic Compute Cloud

VPC : Virtual Private Cloud

CPU : Central Processing Unit

IT : Information Technology

RAM : Random Access Memory

SSH : Secure Shell

LTS : Long Term Support

HTTP : Hypertext Transfer Protocol

HTTPS : Hypertext Transfer Protocol Secure

RDP : Remote Desktop Protocol

TCP : Transmission Control Protocol

CIDR : Classless Inter-Domain Routing

.

Introduction générale

Avec l'évolution rapide des technologies numériques, les infrastructures informatiques connaissent une transformation majeure, notamment grâce à l'essor du **cloud computing**. Ce modèle permet de déployer des ressources informatiques à la demande, tout en offrant une grande flexibilité, une réduction des coûts matériels et une meilleure accessibilité des services. Parmi les plateformes cloud les plus utilisées figure (**AWS**), qui propose une large gamme de services pour la création et la gestion d'environnements informatiques virtuels.

Cependant, la mise en place d'une infrastructure cloud ne se limite pas au déploiement de machines virtuelles. Il est indispensable d'assurer la **surveillance et le contrôle des systèmes** afin de garantir leur bon fonctionnement, leur disponibilité et leurs performances. La supervision permet notamment de détecter rapidement les anomalies, de suivre l'utilisation des ressources (CPU, mémoire, disque, réseau) et d'anticiper d'éventuelles pannes.

Dans ce cadre, des outils de monitoring comme **Zabbix** jouent un rôle central. Zabbix est une solution de supervision open source largement utilisée dans les environnements professionnels pour surveiller des serveurs, des services et des applications, aussi bien sous **Linux** que sous **Windows**. Son intégration avec des technologies de conteneurisation telles que **Docker** permet de simplifier son déploiement et d'améliorer la portabilité de l'infrastructure.

L'objectif de ce projet est de concevoir et de mettre en œuvre une **infrastructure de supervision centralisée dans le cloud AWS**, basée sur un serveur **Zabbix déployé avec Docker**, capable de surveiller un parc informatique composé de machines Linux et Windows. Ce travail permet de mettre en pratique des notions essentielles telles que la création (VPC), la gestion des instances EC2, la sécurisation des accès et la configuration d'agents de supervision.

Les principaux **outils utilisés** dans ce projet sont :

- **AWS** (EC2, VPC, Security Groups) pour l'infrastructure cloud,
- **Docker et Docker Compose** pour la conteneurisation et l'orchestration des services,
- **Zabbix** pour la supervision et le monitoring des systèmes.

À travers ce projet, nous cherchons à développer des compétences pratiques en **cloud computing**, en **administration systèmes** et en **monitoring**, tout en nous rapprochant des pratiques utilisées dans les environnements professionnels réels.

- **Lien vers le dépôt GitHub du projet :**

<https://github.com/elyachioui-douaa/Projet-Mise-en-uvre-d-une-infrastructure-cloud-de-supervision-centralis-e-sous-AWS>

Partie 1

Présentation de l'environnement AWS (Architecture réseau sous AWS)

Présentation de l'environnement AWS (Architecture réseau)

Accès au Learner Lab AWS

L'accès au Learner Lab AWS constitue la première étape essentielle pour la réalisation de ce projet. Le Learner Lab offre un environnement cloud pédagogique permettant de déployer et de manipuler des ressources AWS tout en respectant des contraintes précises en termes de budget, de région et de types d'instances.

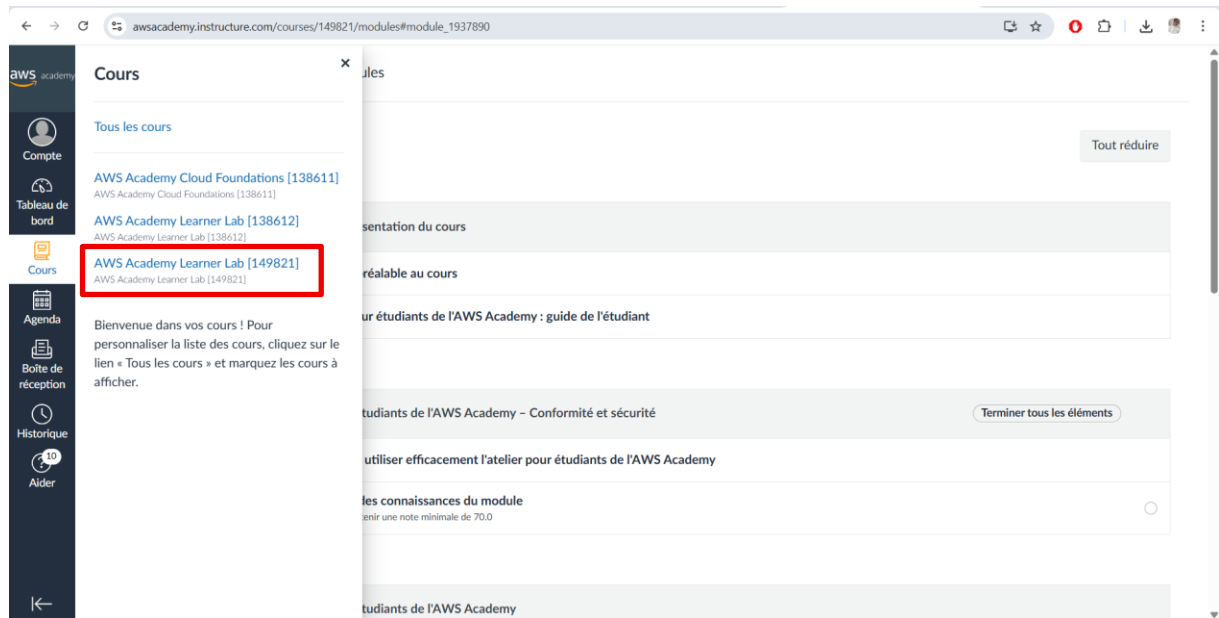


Figure 1: Accès au Learner Lab AWS

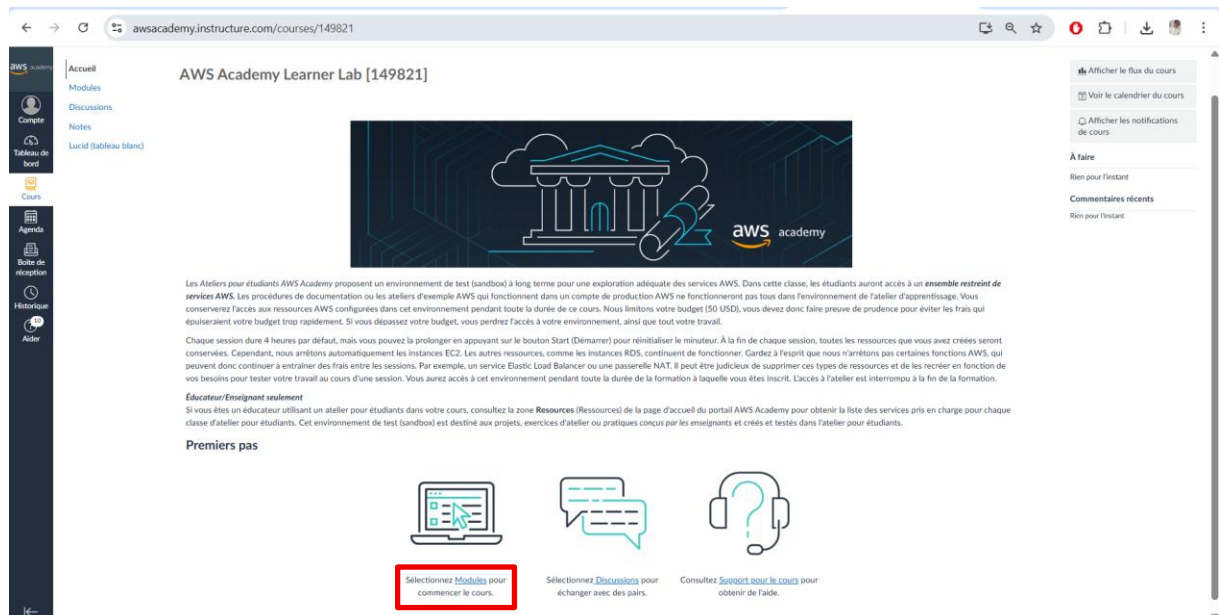
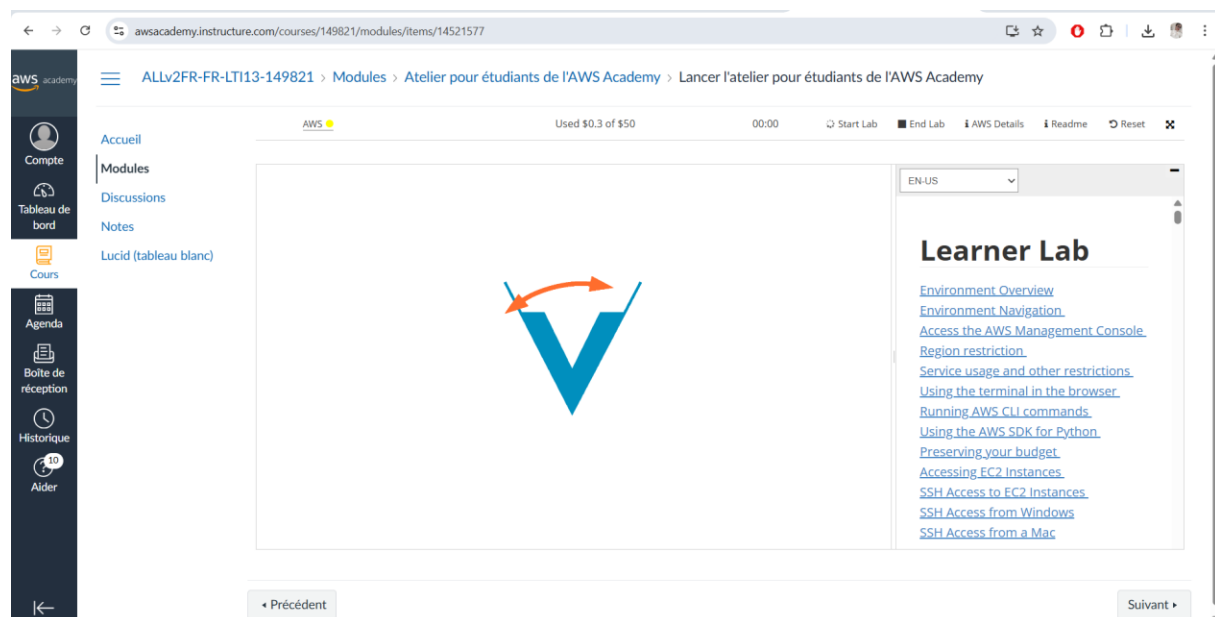
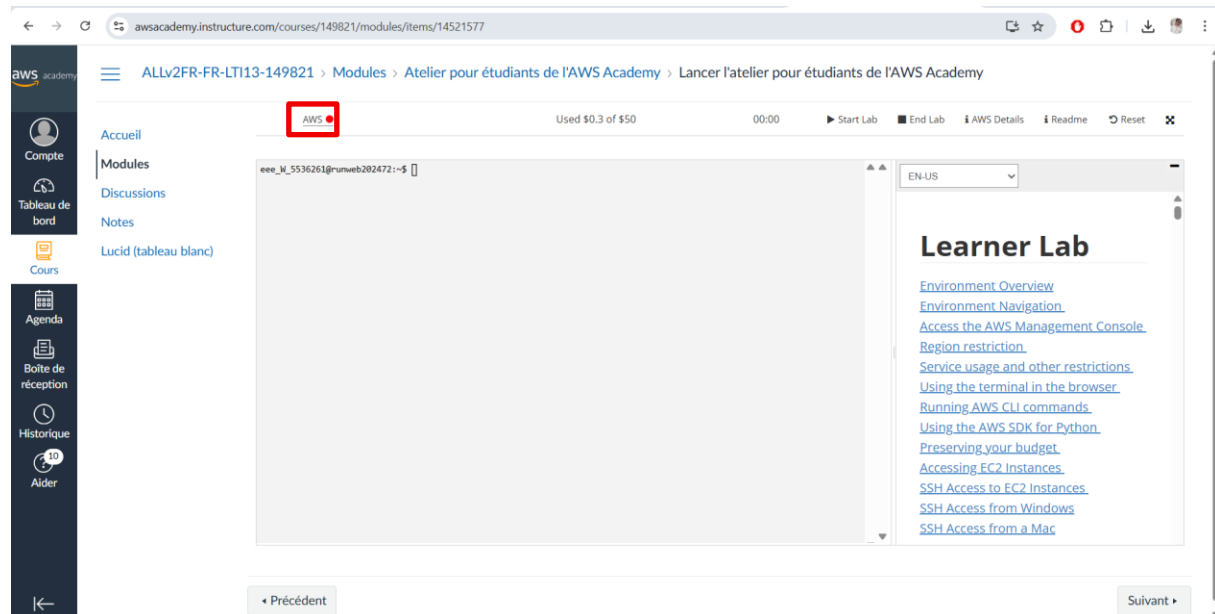


Figure 2: Accès au Learner Lab AWS (Modules)

Dans un premier temps, une connexion a été établie via la plateforme de formation donnant accès au Learner Lab AWS. Une fois le laboratoire lancé, l'accès à la console de gestion AWS a été effectué à l'aide des identifiants temporaires fournis par le Lab.



Après la connexion réussie, les points suivants ont été vérifiés :

- Le démarrage effectif du Lab (statut actif),
- La disponibilité du budget alloué.

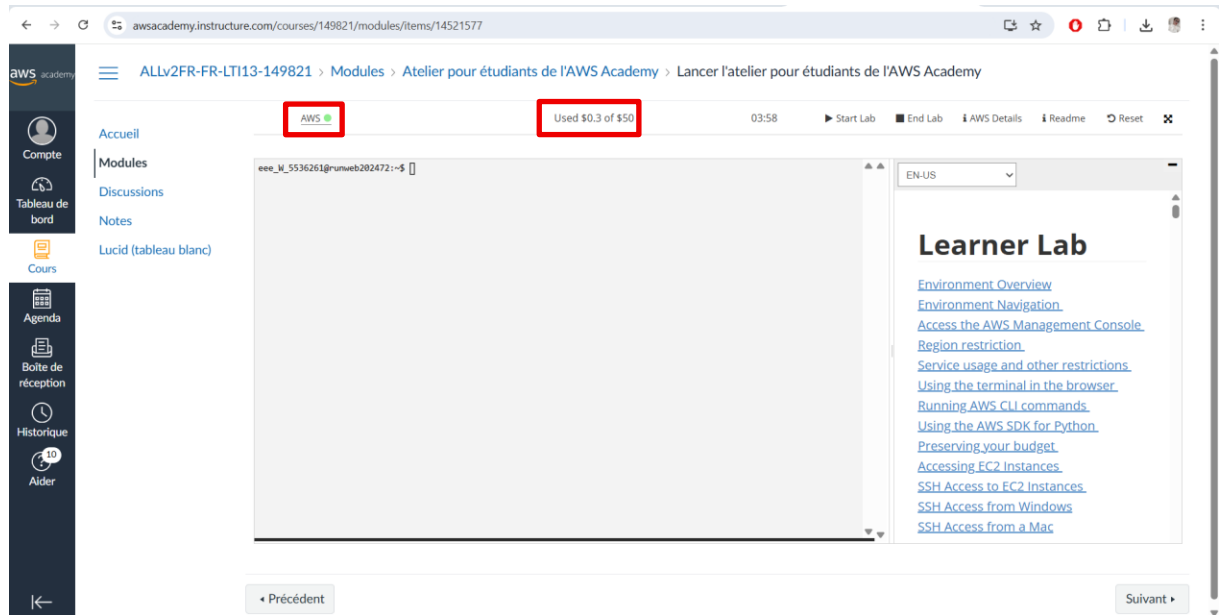


Figure 3: La connexion réussie du Lab

Choix de la région AWS

La sélection de la **région us-east-1 (N. Virginia)**, recommandée pour assurer la stabilité et la compatibilité des services AWS dans le cadre du laboratoire.

Cette étape est primordiale car toute mauvaise configuration initiale (région incorrecte ou Lab inactif) peut entraîner des erreurs lors du déploiement des ressources EC2 et des services réseau ultérieurs.

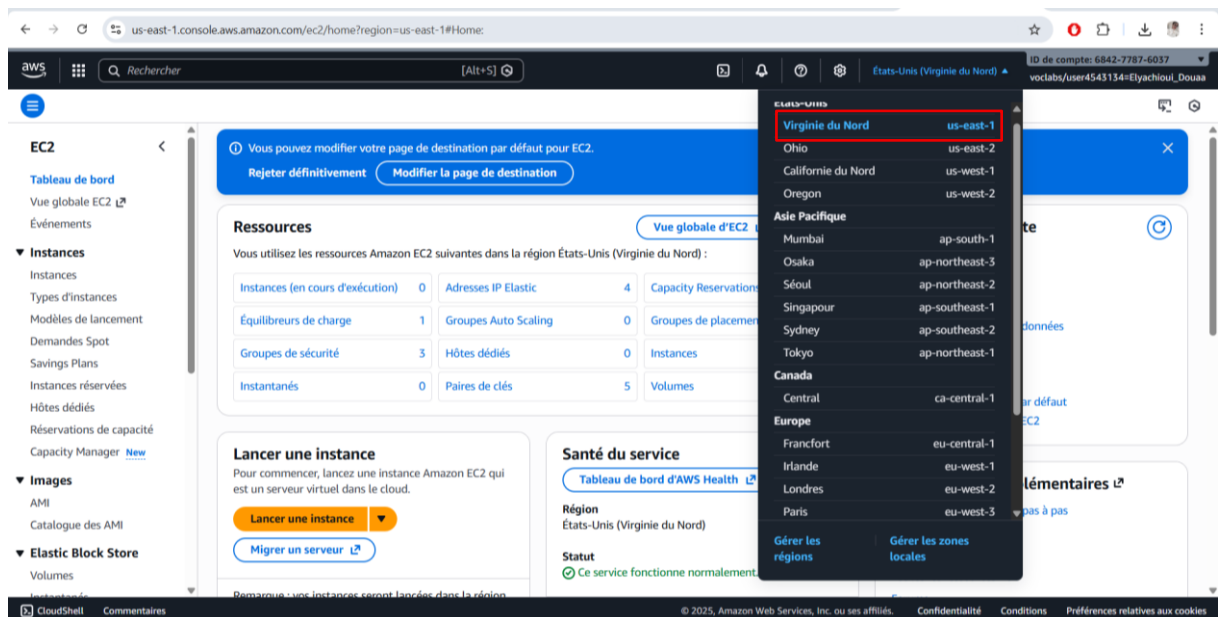


Figure 4: La sélection de la région us-east-1 (N. Virginia)

Création du VPC

Après l'accès réussi au Learner Lab AWS, la deuxième étape a consisté à créer un (VPC) afin d'isoler et structurer l'infrastructure réseau du projet. Le VPC permet de définir un environnement réseau virtuel dans lequel seront déployées les différentes instances EC2.

Dans la console AWS, le service VPC a été sélectionné, puis l'option Create VPC a été utilisée pour créer un nouveau réseau virtuel personnalisé.

Un VPC unique a été mis en place, conformément aux contraintes du Learner Lab, afin de simplifier l'architecture et la gestion des ressources.

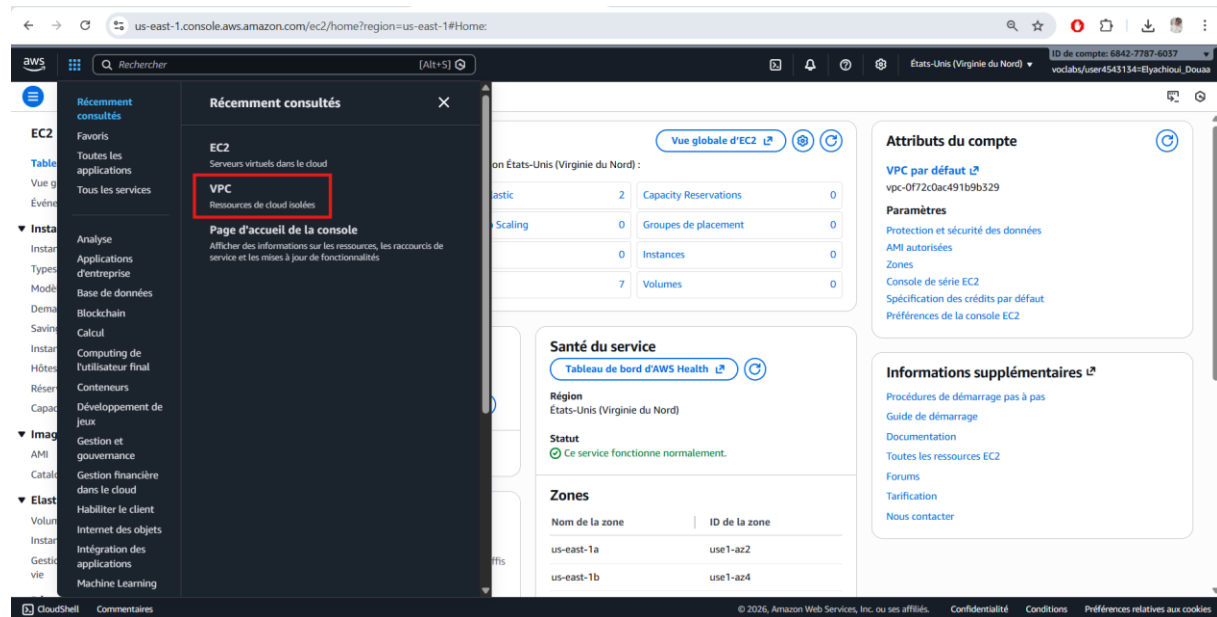
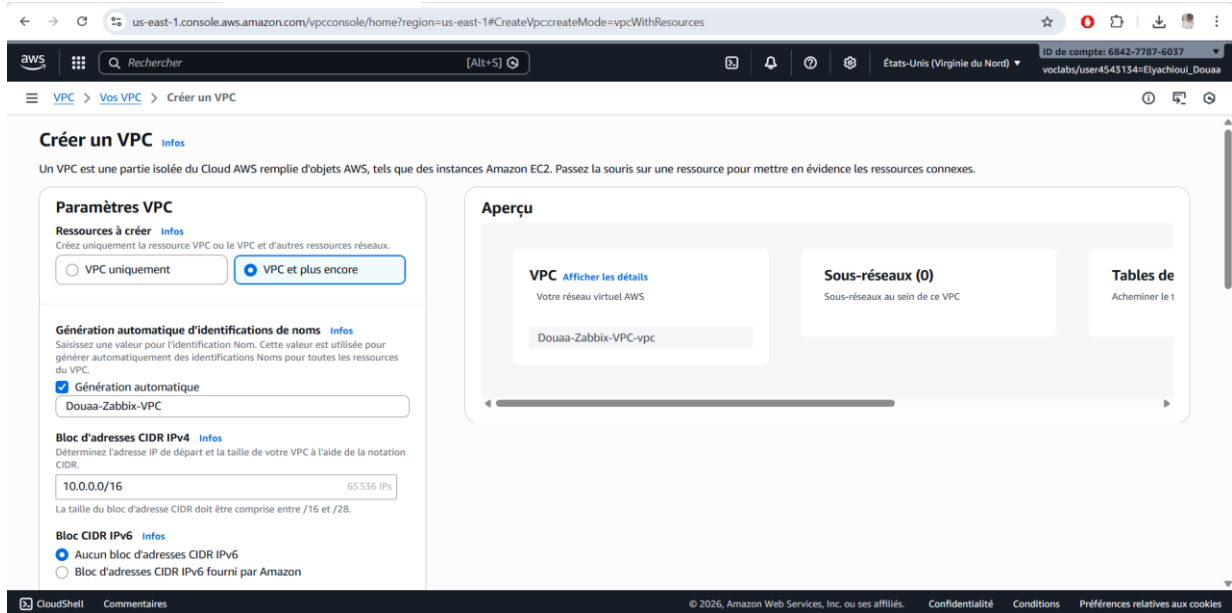


Figure 5:Création du VPC

Les paramètres principaux configurés lors de la création du VPC sont les suivants :

- Nom du VPC : *Douaa-Zabbix-VPC*,
- Plage d'adresses IPv4 (CIDR) : 10.0.0.0/16,
- Support DNS : activé,
- Résolution DNS : activée.



Créer un VPC Infos

Un VPC est une partie isolée du Cloud AWS remplie d'objets AWS, tels que des instances Amazon EC2. Passez la souris sur une ressource pour mettre en évidence les ressources connexes.

Paramètres VPC

Ressources à créer Infos
 Créez uniquement la ressource VPC ou le VPC et d'autres ressources réseaux.

☐ VPC uniquement ☒ VPC et plus encore

Génération automatique d'identifications de noms Infos
 Saisissez une valeur pour l'identification Nom. Cette valeur est utilisée pour générer automatiquement des identifications Noms pour toutes les ressources du VPC.

☒ Génération automatique
 Douaa-Zabbix-VPC

Bloc d'adresses CIDR IPv4 Infos
 Déterminez l'adresse IP de départ et la taille de votre VPC à l'aide de la notation CIDR.

10.0.0.0/16 65 536 IPs

La taille du bloc d'adresse CIDR doit être comprise entre /16 et /28.

Bloc CIDR IPv6 Infos
☒ Aucun bloc d'adresses CIDR IPv6
☐ Bloc d'adresses CIDR IPv6 fourni par Amazon

Aperçu

VPC Afficher les détails
 Votre réseau virtuel AWS

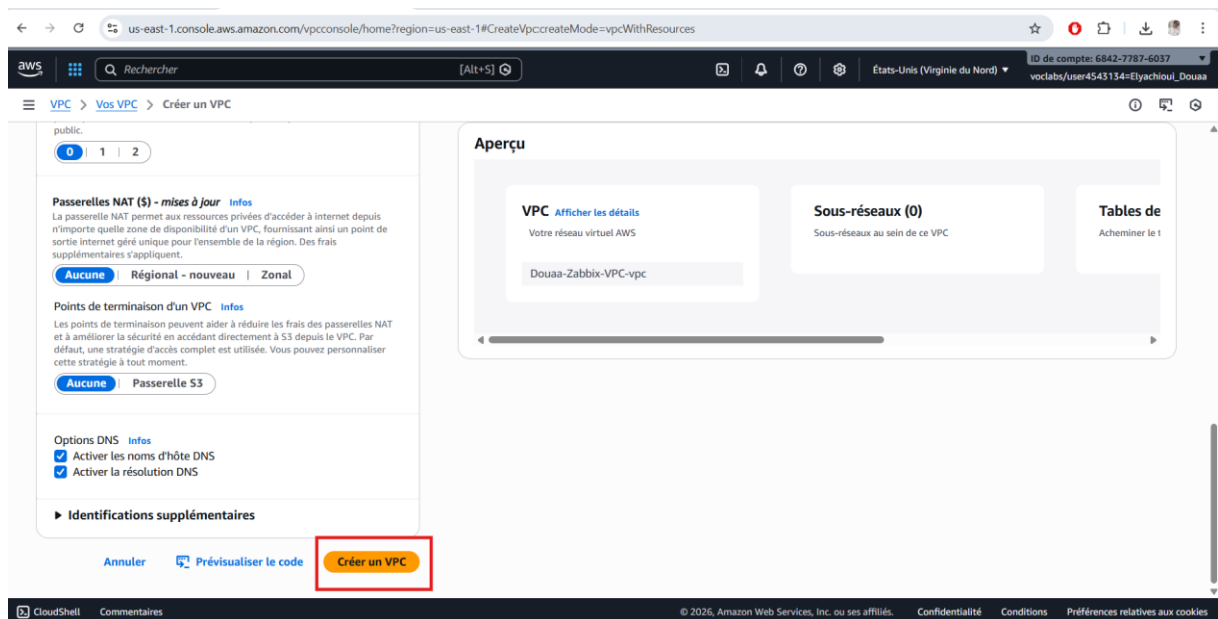
Douaa-Zabbix-VPC

Sous-réseaux (0)
 Sous-réseaux au sein de ce VPC

Tables de
 Acheminer le 1

Figure 6: Les paramètres principaux configurés lors de la création du VPC

L'activation des fonctionnalités DNS est indispensable pour permettre aux instances EC2 de communiquer efficacement entre elles et avec les services AWS.



Passerelles NAT (\$) - mises à jour Infos
 La passerelle NAT permet aux ressources privées d'accéder à Internet depuis n'importe quelle zone de disponibilité d'un VPC, fournissant ainsi un point de sortie Internet géré unique pour l'ensemble de la région. Des frais supplémentaires s'appliquent.

Aucune Régional - nouveau Zonal

Points de terminaison d'un VPC Infos
 Les points de terminaison peuvent aider à réduire les frais des passerelles NAT et à améliorer la sécurité en accédant directement à S3 depuis le VPC. Par défaut, une stratégie d'accès complet est utilisée. Vous pouvez personnaliser cette stratégie à tout moment.

Aucune Passerelle S3

Options DNS Infos
☒ Activer les noms d'hôte DNS
☒ Activer la résolution DNS

► Identifications supplémentaires

Annuler Prévisualiser le code **Créer un VPC**

Figure 7: L'activation des fonctionnalités DNS

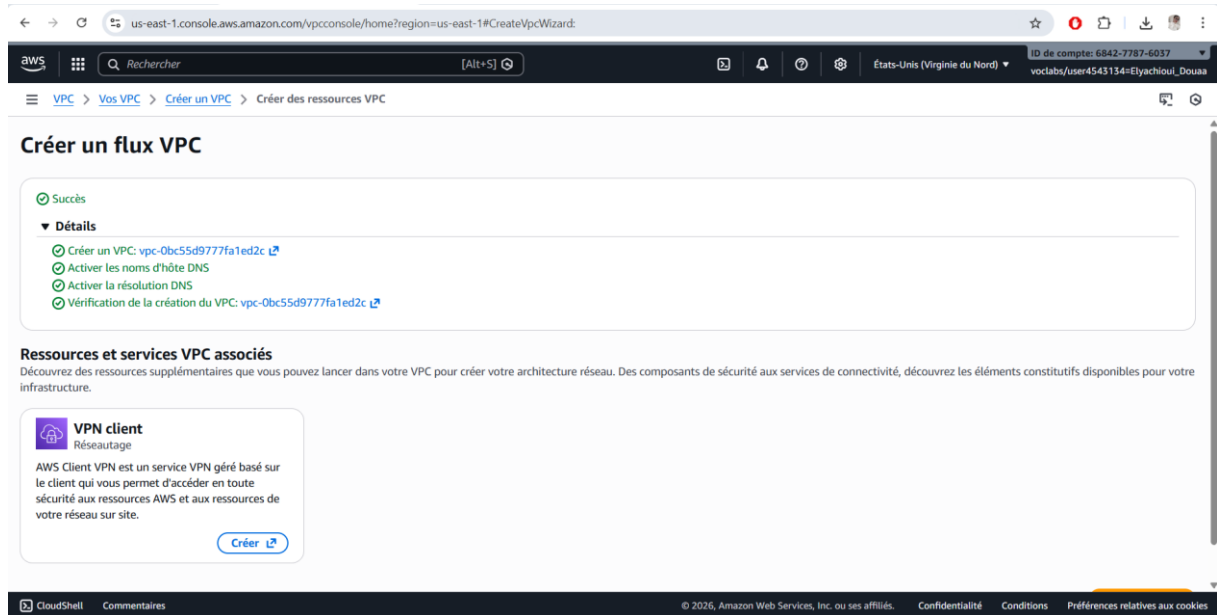


Figure 8: Création du VPC dédié à l'infrastructure de supervision Zabbix

Cette étape constitue la base de toute l'architecture réseau du projet, car toutes les ressources (instances EC2, sous-réseaux et groupes de sécurité) seront associées à ce VPC.

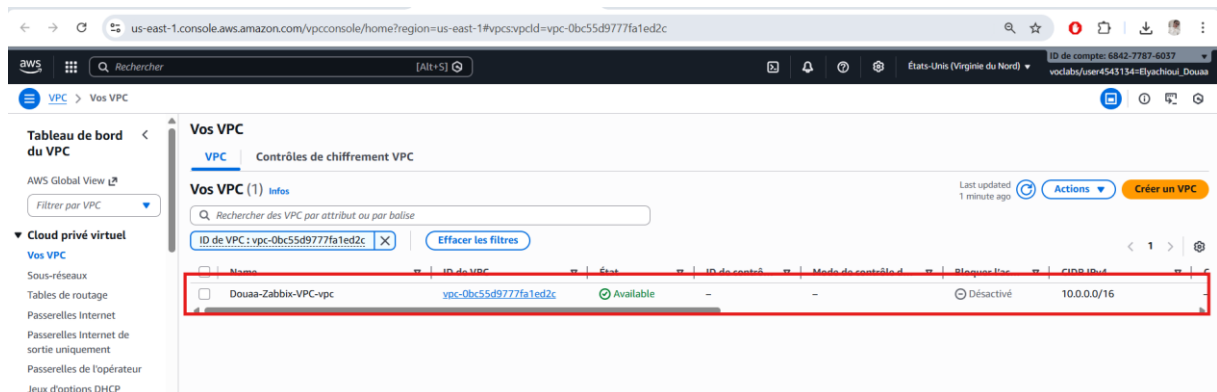


Figure 9: VPC

Création et configuration de la Passerelle Internet (IGW)

Pour permettre l'accès à Internet aux ressources déployées dans le VPC, il est nécessaire de mettre en place une **passerelle Internet (Internet Gateway)**. Cette passerelle assure la communication entre le réseau virtuel AWS (VPC) et le réseau Internet public.

Dans la console de gestion AWS, le service **VPC** a été sélectionné, puis l'option **Internet Gateways** a été utilisée afin de créer une nouvelle passerelle. Une fois créée, cette passerelle a été **attachée au VPC** précédemment configuré.

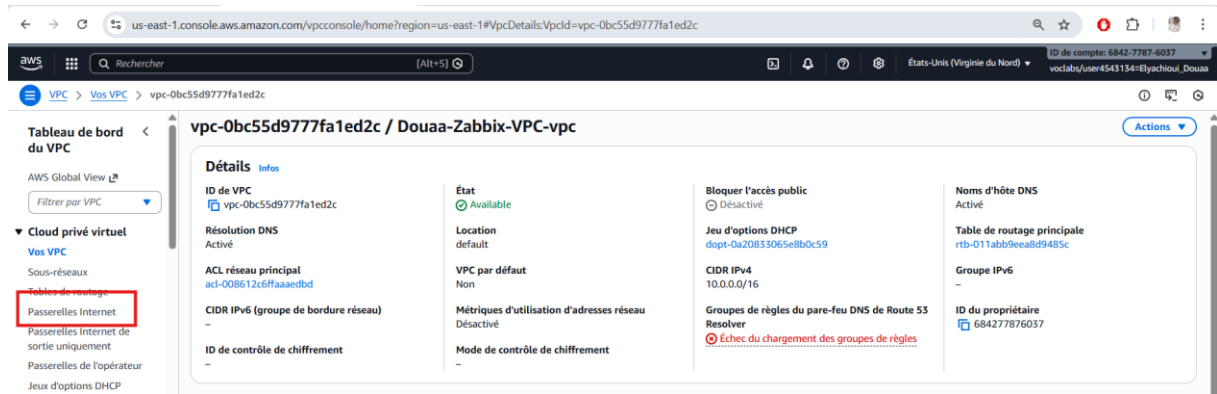
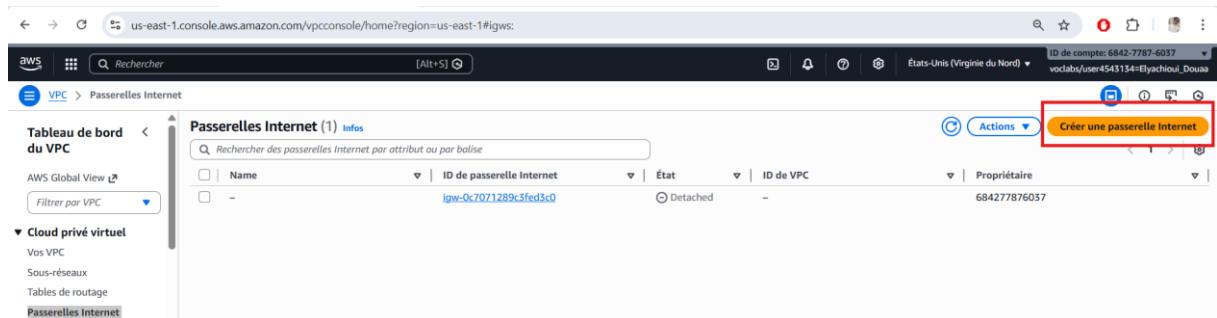


Figure 10: Etape 1- Passerelle Internet



Les paramètres suivants ont été définis :

- **Nom de la passerelle Internet** : *Douaa-Zabbix-IGW*,
- **VPC associé** : *Douaa-Zabbix-VPC*.

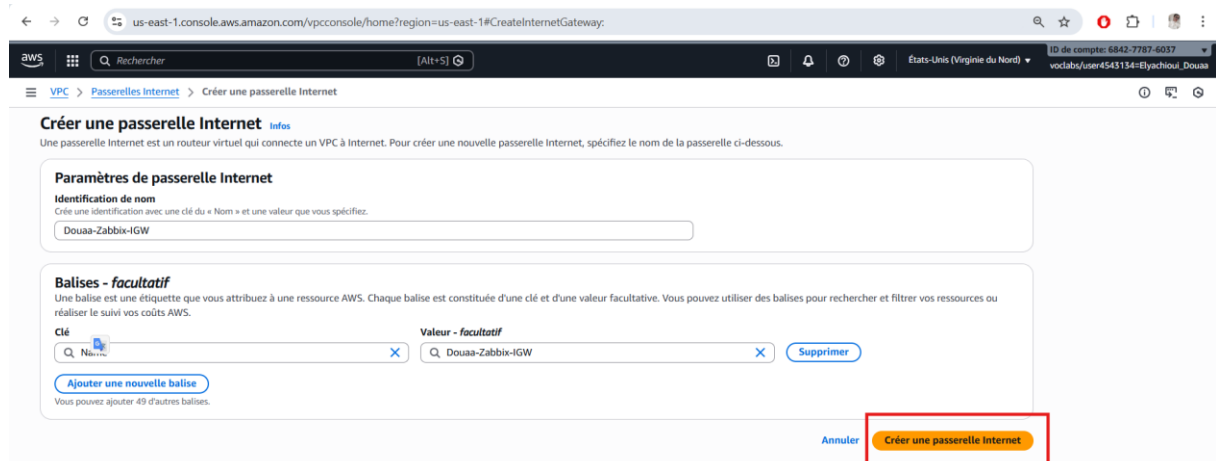


Figure 11: Etape 2- Créer une Passerelle Internet

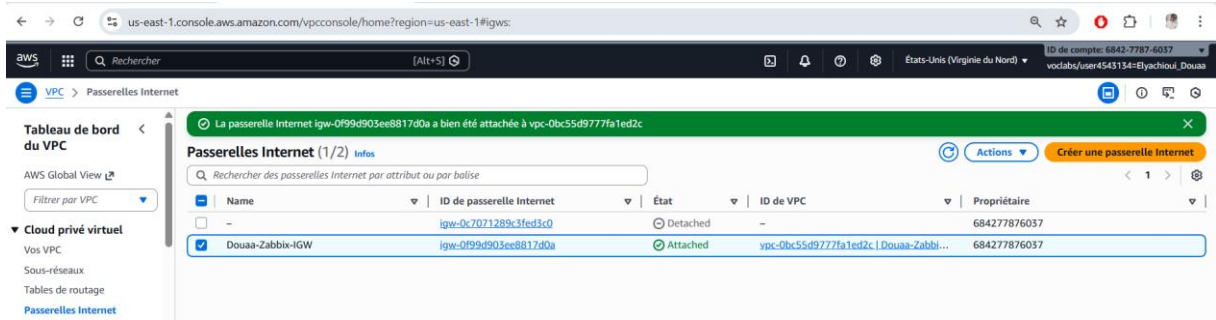


Figure 12: Création réussie de la passerelle Internet (Internet Gateway)

Après l'attachement de la passerelle au VPC, une **table de routage** a été configurée afin de permettre le trafic sortant vers Internet. Une route par défaut a été ajoutée avec les paramètres suivants :

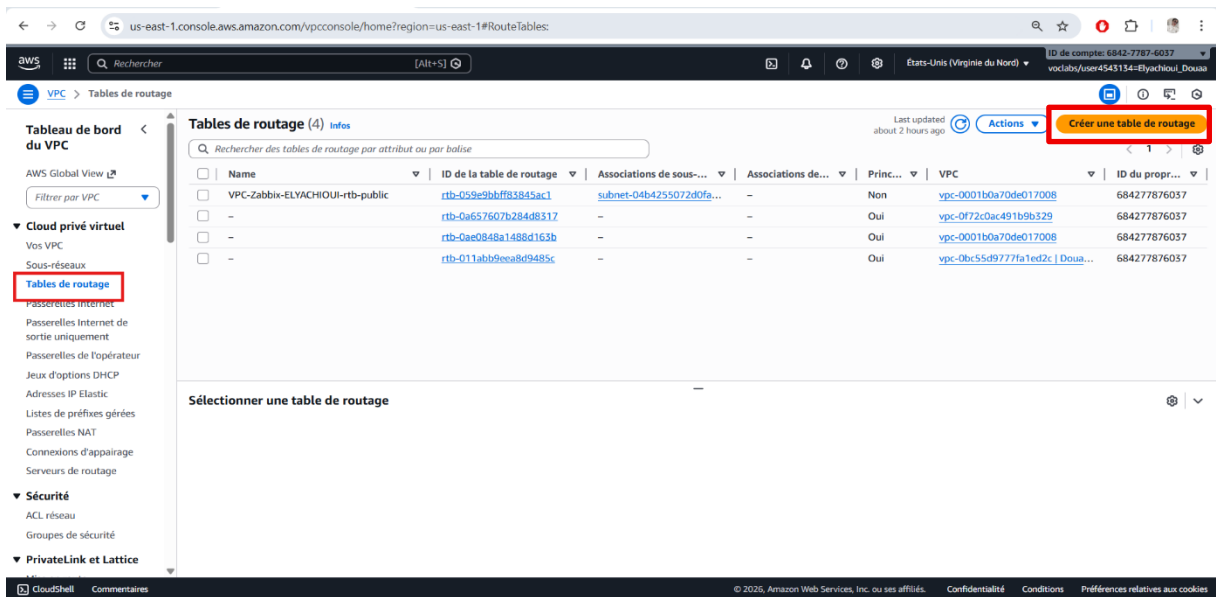


Figure 13: Créer une table de routage

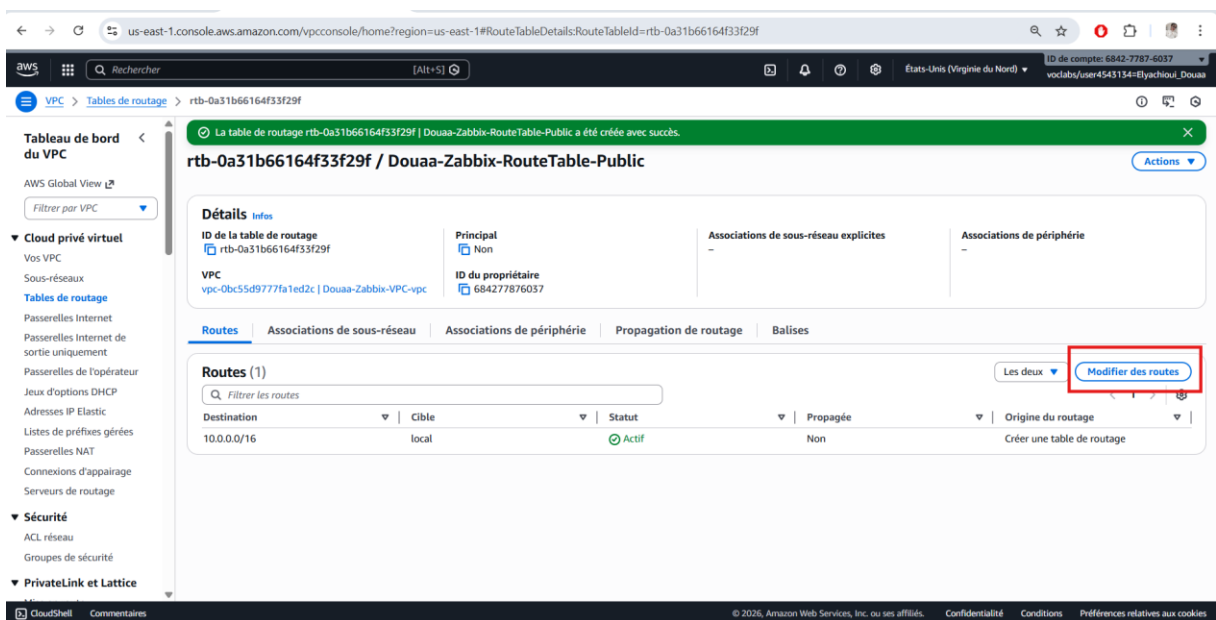


Figure 14: Création réussie de la table de routage associée au sous-réseau public

- **Destination** : 0.0.0.0/0,
- **Cible (Target)** : *Internet Gateway (Douaa-Zabbix-IGW)*.

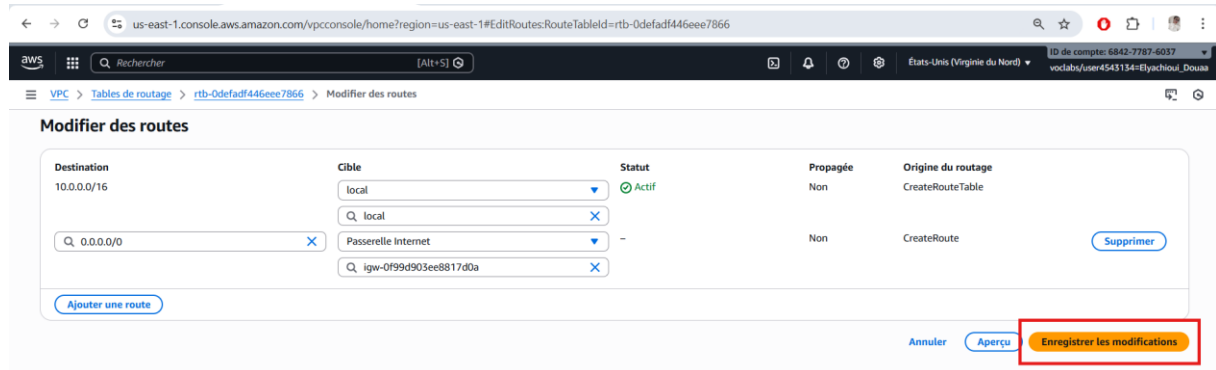


Figure 15: Modification des routes de la table de routage

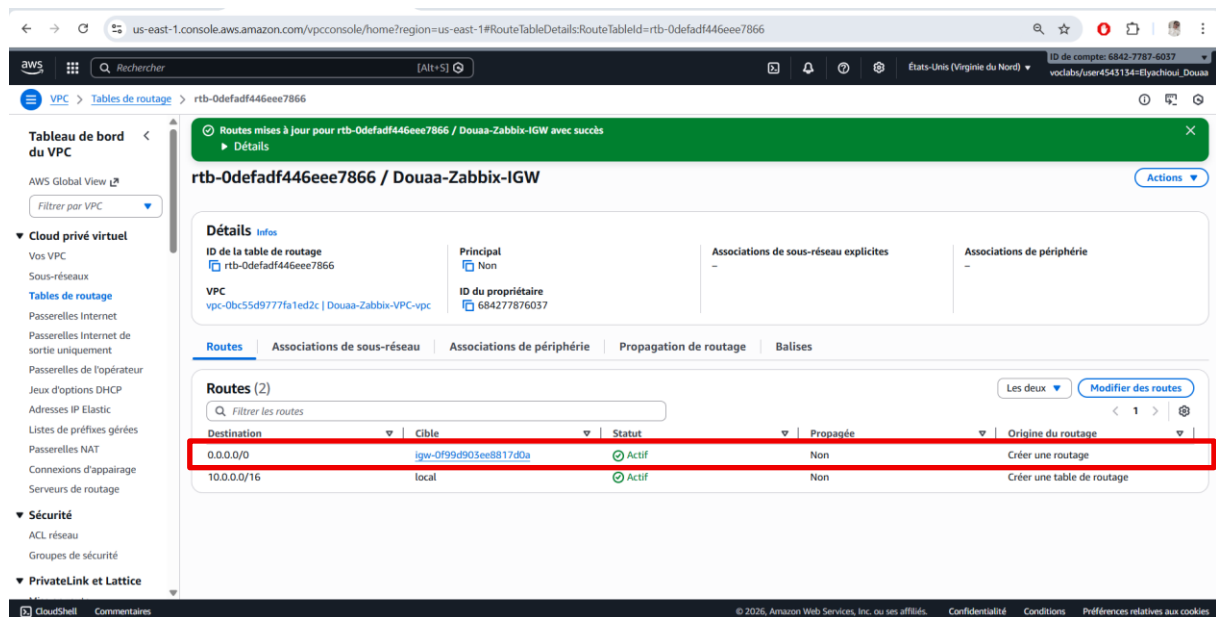


Figure 16: Succès de la modification de la table de routage

Cette table de routage a ensuite été associée au **sous-réseau public**, garantissant ainsi que toutes les instances EC2 déployées dans ce sous-réseau disposent d'un accès Internet fonctionnel.

Cette étape est indispensable pour :

- l'installation des dépendances système,
- le téléchargement des images Docker,
- l'accès à l'interface Web de Zabbix depuis un navigateur externe.

Création d'un Sous-Réseau Public

Après la mise en place du VPC et de la passerelle Internet, l'étape suivante a consisté à créer un **sous-réseau public** destiné à héberger les différentes instances EC2 du projet (serveur Zabbix, client Linux et client Windows).

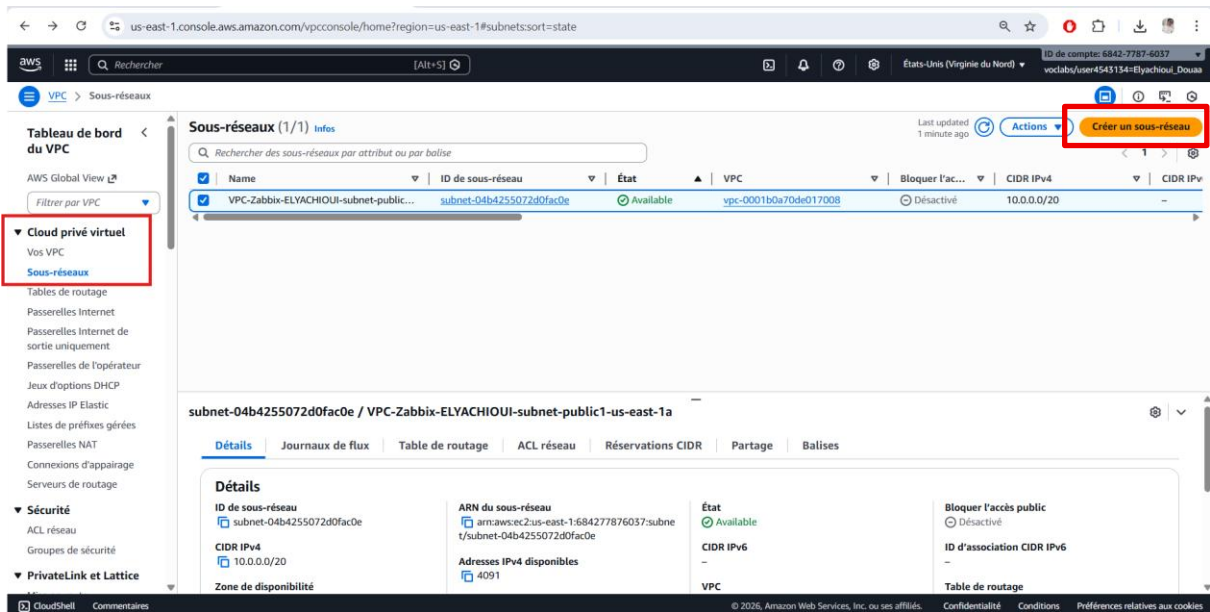


Figure 17:Cr er un sous-r seau public

Les param tres de configuration suivants ont  t  d finis :

- **Nom du sous-r seau** : *Douaa-Zabbix- Subnet-Public*,
- **VPC associ ** : *Douaa-Zabbix-VPC*,
- **Zone de disponibilit ** : *us-east-1a*,
- **Plage d'adresses IPv4 (CIDR)** : *10.0.1.0/24*.

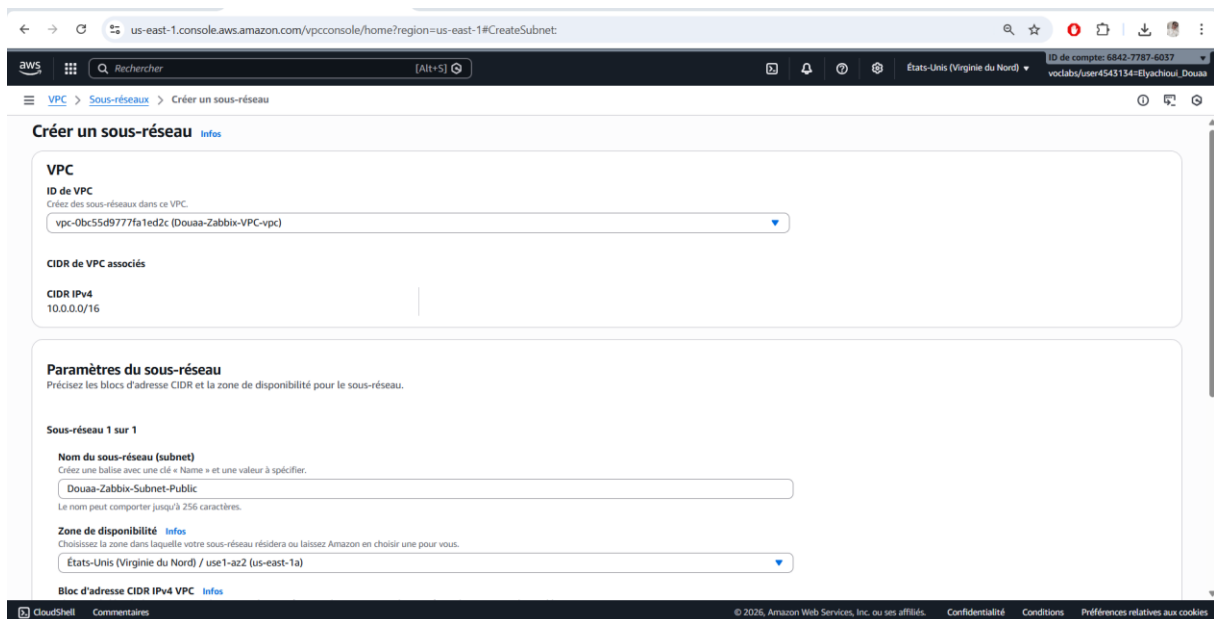
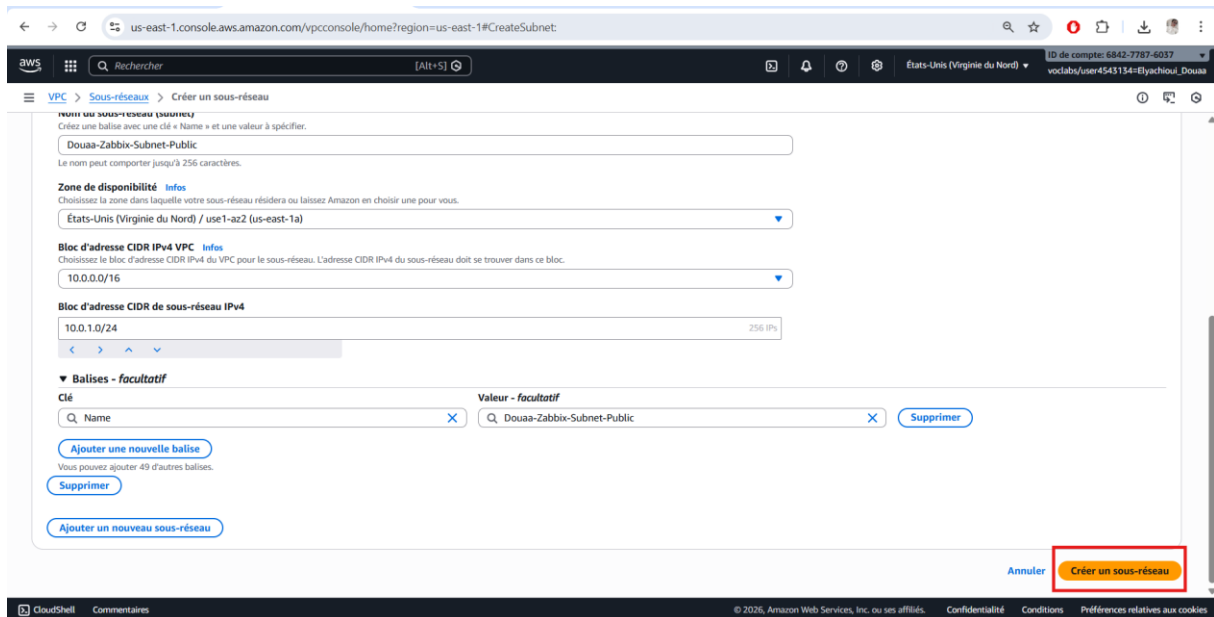


Figure 18:Les param tres de configuration d'un sous- r seau



us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#CreateSubnet

VPC > **Sous-réseau** > Créer un sous-réseau

Nom du sous-réseau (obligatoire)
 Créez une balise avec une clé « Name » et une valeur à spécifier.
 Douaa-Zabbix-Subnet-Public

Le nom peut comporter jusqu'à 256 caractères.

Zone de disponibilité **Infos**
 Choisissez la zone dans laquelle votre sous-réseau résidera ou laissez Amazon en choisir une pour vous.
 États-Unis (Virginie du Nord) / us-east-1a

Bloc d'adresse CIDR IPv4 VPC **Infos**
 Choisissez le bloc d'adresse CIDR IPv4 du VPC pour le sous-réseau. L'adresse CIDR IPv4 du sous-réseau doit se trouver dans ce bloc.
 10.0.0.0/16

Bloc d'adresse CIDR de sous-réseau IPv4
 10.0.1.0/24 256 IPs

Balises - facultatif

Clé Valeur - facultatif

Q Name X Q Douaa-Zabbix-Subnet-Public X Supprimer

Ajouter une nouvelle balise

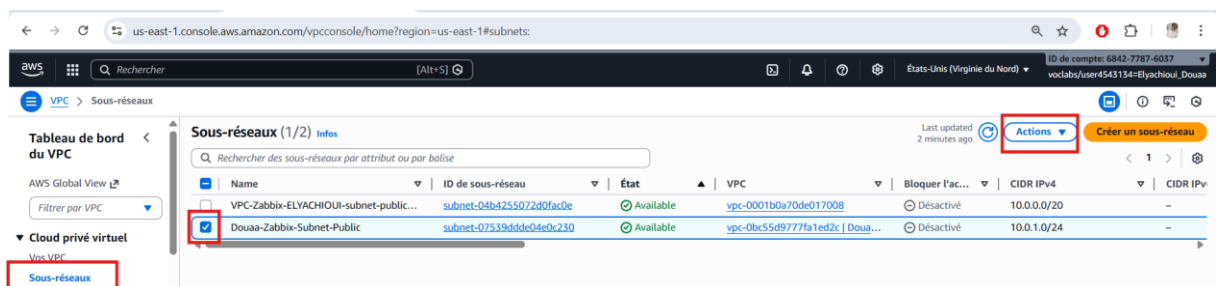
Vous pouvez ajouter 49 d'autres balises.

Supprimer

Ajouter un nouveau sous-réseau

Annuler **Créer un sous-réseau**

Afin de rendre ce sous-réseau public, l'option **Auto-assign public IPv4 address** a été activée. Cette configuration permet d'attribuer automatiquement une adresse IP publique aux instances EC2 lancées dans ce sous-réseau, facilitant ainsi l'accès distant via **SSH**, **RDP** et **navigateur Web**.



us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#subnets

VPC > **Sous-réseau**

Tableau de bord du VPC

AWS Global View

Filter par VPC

Cloud privé virtuel

Vos VPC

Sous-réseau

Sous-réseaux (1/2) **Infos**

Rechercher des sous-réseaux par attribut ou par balise

Last updated 2 minutes ago **Actions** **Créer un sous-réseau**

	Name	ID de sous-réseau	État	VPC	Bloquer l'accès	CIDR IPv4	CIDR IPv6
☐	VPC-Zabbix-ELYACHIOUI-subnet-public...	subnet-04b4255072d0fac0e	Available	vpc-0001b0a70de017008	Désactivé	10.0.0.0/20	-
☒	Douaa-Zabbix-Subnet-Public	subnet-07539dddc04e0c230	Available	vpc-0bc55d9777fa1ed2c Doua...	Désactivé	10.0.1.0/24	-

Last updated 4 minutes ago **Actions**

- Afficher les détails
- Créer un journal de flux
- Modifier les paramètres de sous-réseau**
- Modifier les blocs CIDR IPv6
- Modifier une association de liste ACL réseau
- Modifier une association des tables de routage
- Modifier les réservations d'adresses CIDR
- Partager le sous-réseau
- Gérer les balises
- Supprimer le sous-réseau

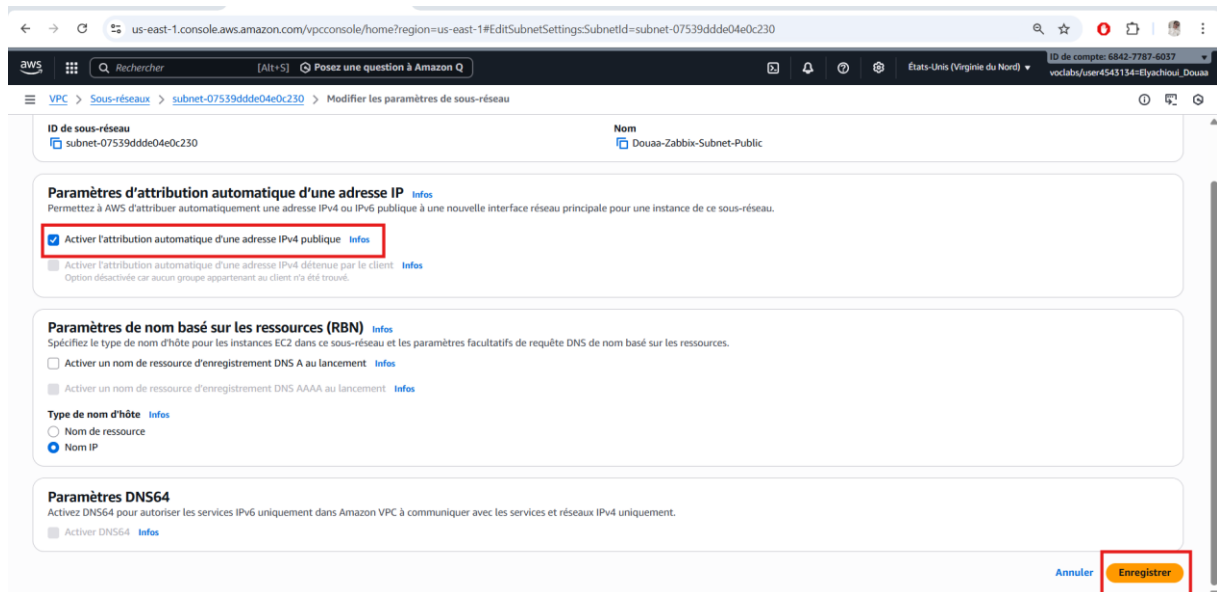


Figure 19: l'option Auto-assign public IPv4 address a été activée

La création de ce sous-réseau public est essentielle pour garantir :

- l'accessibilité des instances depuis l'extérieur,
- la communication avec Internet,
- le respect des contraintes du Learner Lab AWS.

Configuration de la Table de Routage



Figure 20: Mappage des ressources

Création et Configuration des Groupes de Sécurité

Afin d'assurer la sécurité de l'infrastructure tout en permettant les communications nécessaires au fonctionnement de Zabbix, deux groupes de sécurité distincts ont été créés : un groupe dédié au serveur Zabbix et un autre dédié aux machines clientes (Linux et Windows).

Cette séparation permet une meilleure organisation, une sécurité renforcée et une gestion plus claire des flux réseau. Les étapes de création sont les suivantes :

Étapes pour créer le Groupe de Sécurité du Serveur Zabbix :

- ❖ Dans la console AWS, sélectionner le service ECE, puis cliquer sur groupe de sécurité.

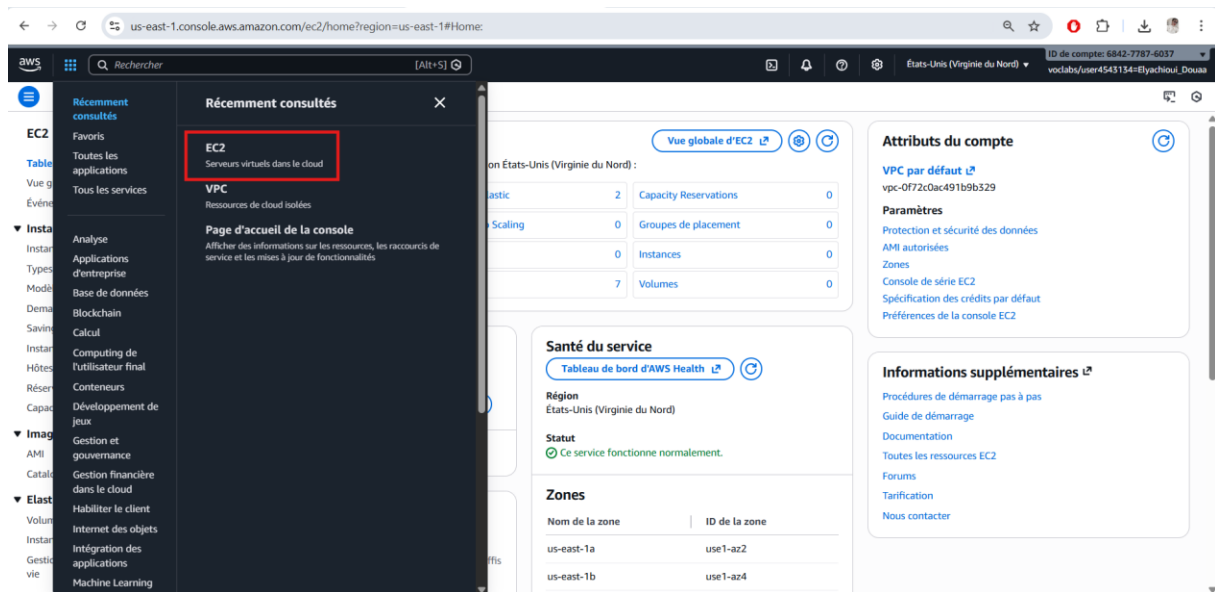


Figure 21: Sélectionner le service ECE

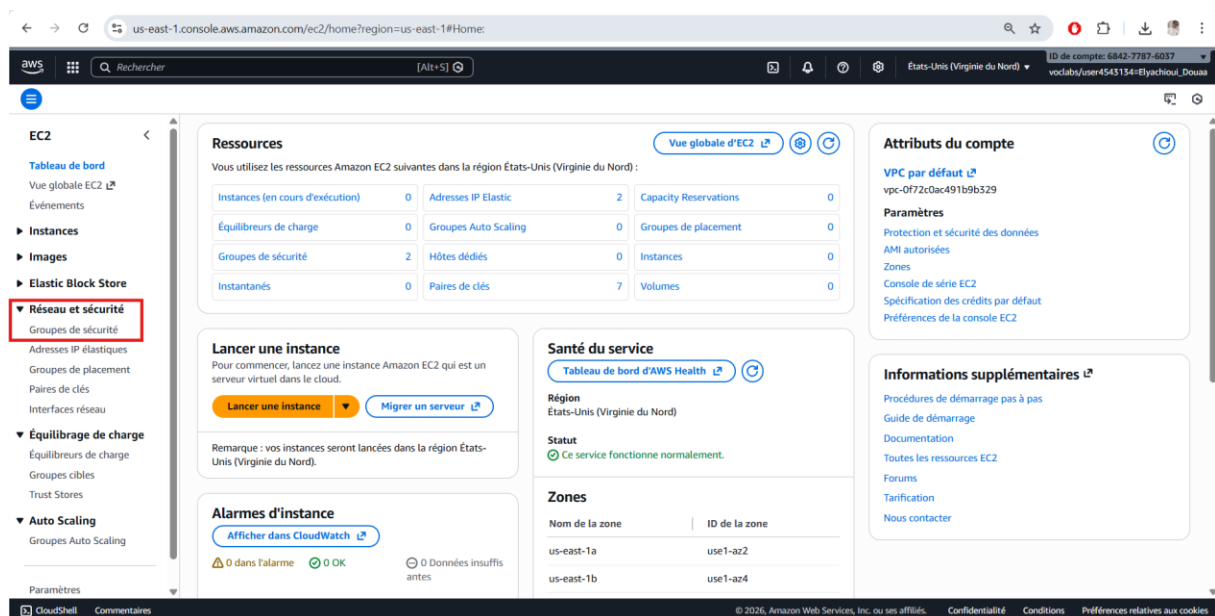


Figure 22: Cliquer sur groupe de sécurité

❖ Cliquer sur Créer un groupe de sécurité

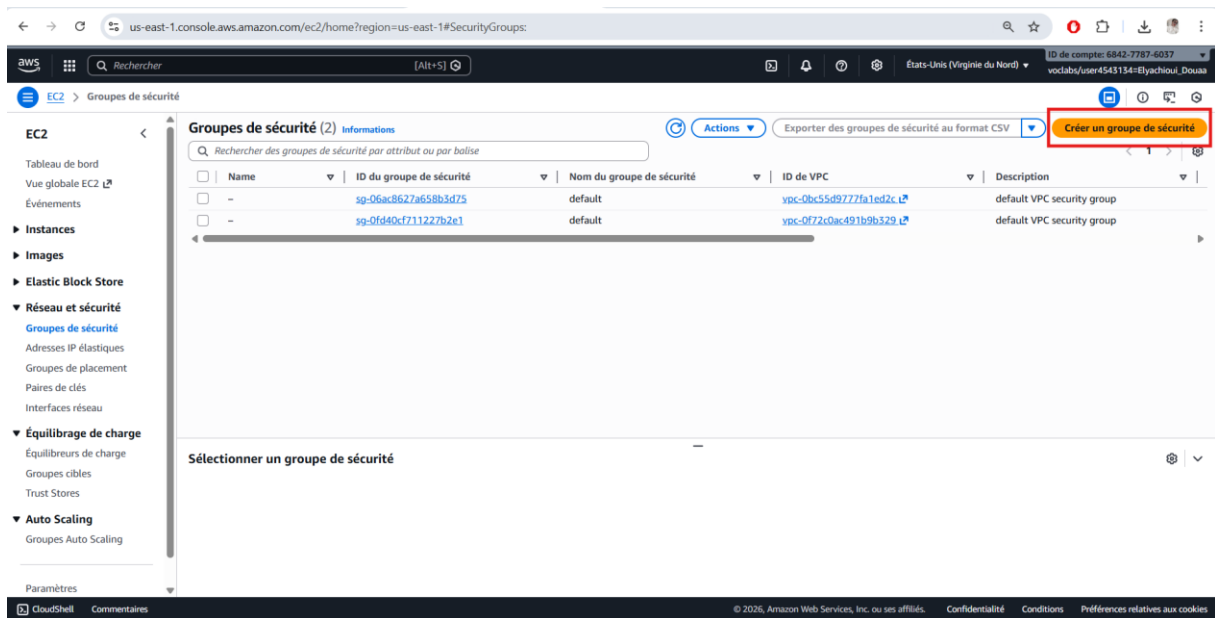
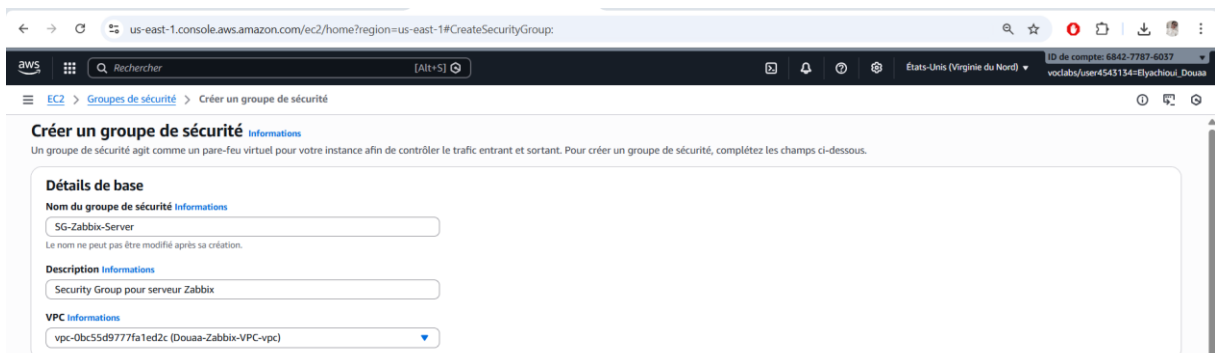


Figure 23: Créer un groupe de sécurité

❖ Saisir les informations suivantes :

- Nom : SG-Zabbix-Server
- Description : Groupe de sécurité pour le serveur Zabbix
- VPC associé : Douaa-Zabbix-VPC



❖ Configurer les règles entrantes (Inbound Rules) :

- SSH : port 22, source = Mon IP
- HTTP : port 80, source = Anywhere (0.0.0.0/0)
- HTTPS : port 443, source = Anywhere
- Custom TCP : port 10051, source = Anywhere (pour les agents Zabbix)

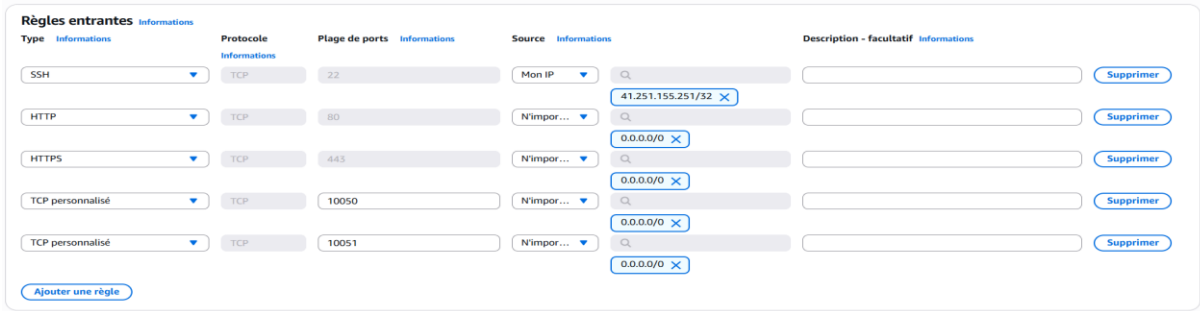
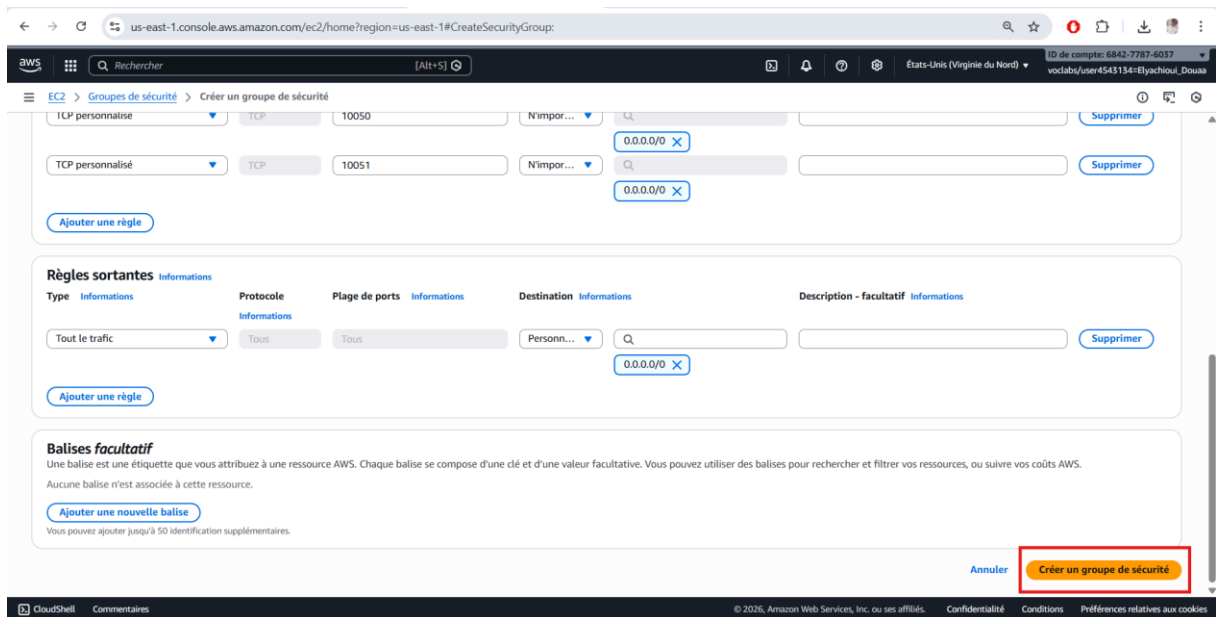


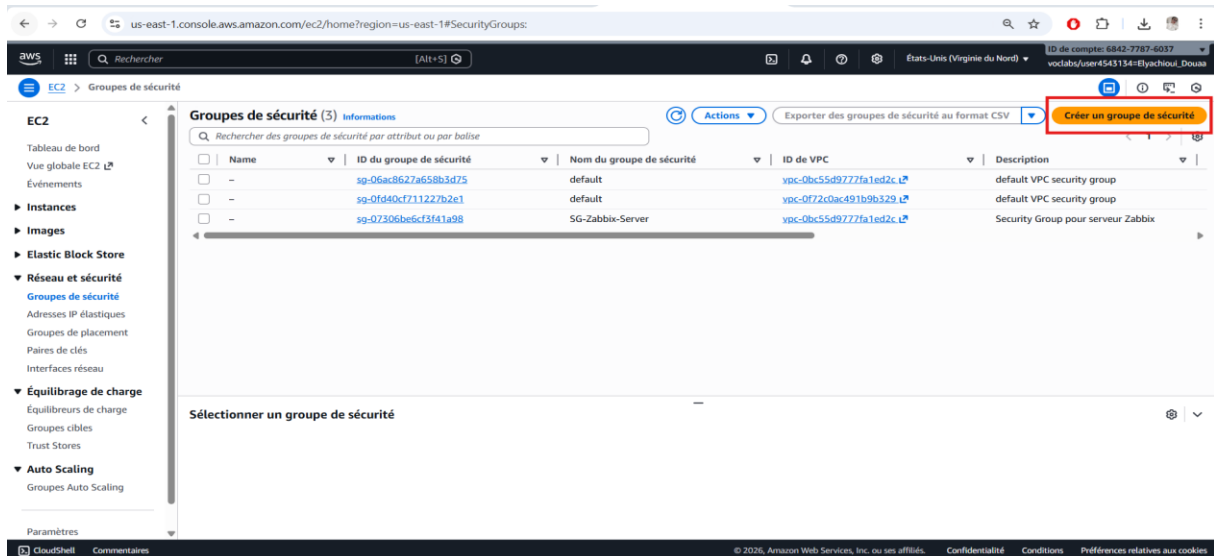
Figure 24: Configurer les règles entrantes

- ❖ Laisser les règles sortantes (Outbound Rules) par défaut pour autoriser tout le trafic sortant.
- ❖ Cliquer sur Créer le groupe de sécurité pour finaliser la création.



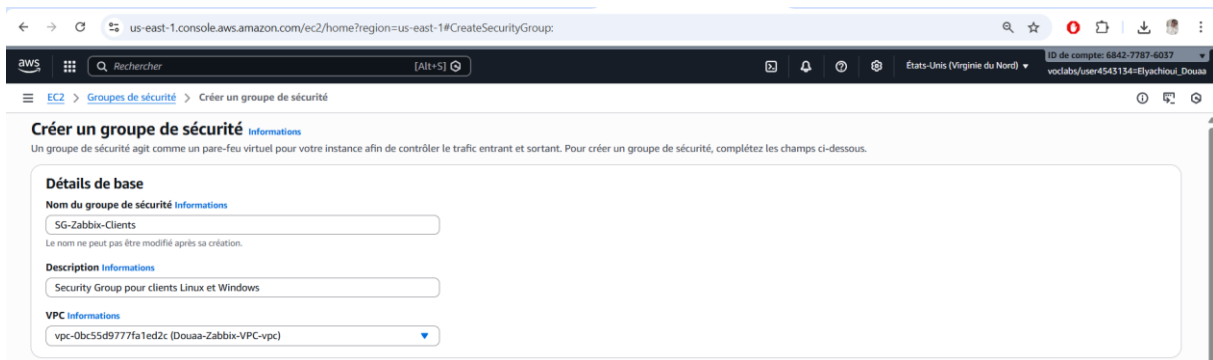
Étapes pour créer le Groupe de Sécurité des Clients

- ❖ Toujours dans la console AWS, sous-groupes de sécurité, cliquer sur Créer un groupe de sécurité.



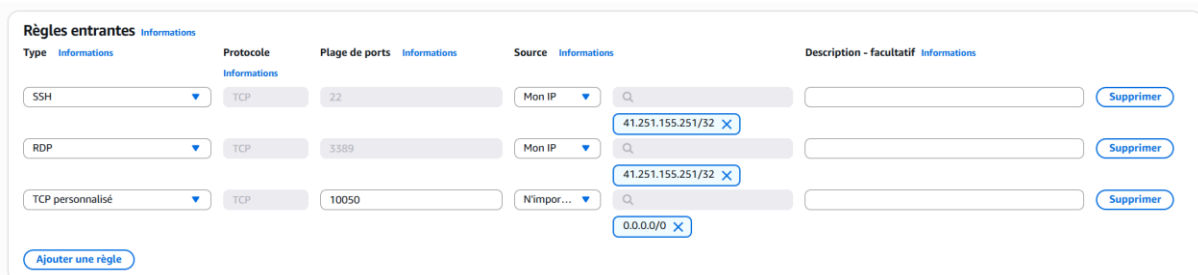
❖ Remplir les informations suivantes :

- Nom : Zabbix-Clients-SG
- Description : Groupe de sécurité pour les machines clientes
- VPC associé : Douaa-Zabbix-VPC



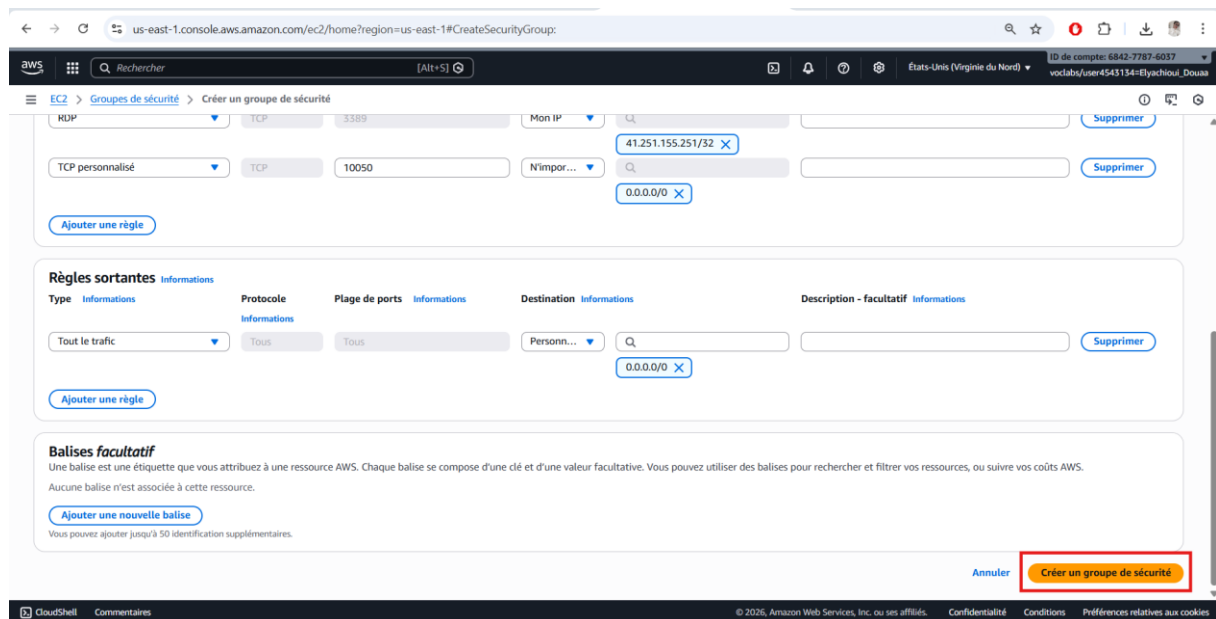
❖ Configurer les règles entrantes (Inbound Rules) :

- SSH : port 22, source = Mon IP (pour accéder au client Linux)
- RDP : port 3389, source = Mon IP (pour accéder au client Windows)
- Custom TCP : port 10050, source = Anywhere (pour que l'agent Zabbix envoie ses données)



❖ Laisser les règles sortantes par défaut pour autoriser le trafic sortant vers le serveur.

❖ Cliquer sur Créer le groupe de sécurité pour finaliser.



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#CreateSecurityGroup:

EC2 > Groupes de sécurité > Créer un groupe de sécurité

RDP TCP 3389 Mon IP 41.251.155.251/32 Supprimer

TCP personnalisé TCP 10050 N'importe quel 0.0.0.0/0 Supprimer

Ajouter une règle

Règles sortantes Informations

Type Informations Protocole Informations Plage de ports Informations Destination Informations Description - facultatif Informations

Tout le trafic Tous Tous Personne 0.0.0.0/0 Supprimer

Ajouter une règle

Balises facultatif

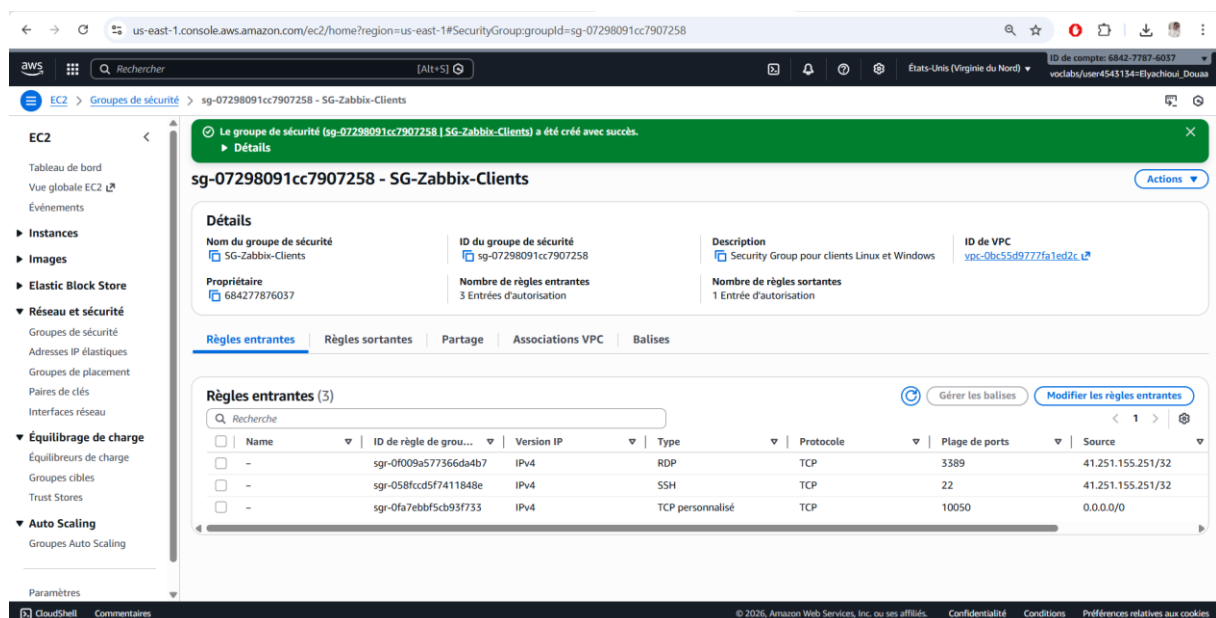
Une balise est une étiquette que vous attribuez à une ressource AWS. Chaque balise se compose d'une clé et d'une valeur facultative. Vous pouvez utiliser des balises pour rechercher et filtrer vos ressources, ou suivre vos coûts AWS.

Aucune balise n'est associée à cette ressource.

Ajouter une nouvelle balise

Vous pouvez ajouter jusqu'à 50 identifications supplémentaires.

Annuler **Créer un groupe de sécurité**



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#SecurityGroup:groupId=sg-07298091cc7907258

EC2 > Groupes de sécurité > sg-07298091cc7907258 - SG-Zabbix-Clients

Le groupe de sécurité (sg-07298091cc7907258 | SG-Zabbix-Clients) a été créé avec succès.

Détails

sg-07298091cc7907258 - SG-Zabbix-Clients Actions

Détails

Nom du groupe de sécurité SG-Zabbix-Clients ID du groupe de sécurité sg-07298091cc7907258 Description Security Group pour clients Linux et Windows ID de VPC vpc-0bc5d977fa1ed2c

Propriétaire 684277876037 Nombre de règles entrantes 3 Entrées d'autorisation Nombre de règles sortantes 1 Entrée d'autorisation

Règles entrantes Règles sortantes Partage Associations VPC Balises

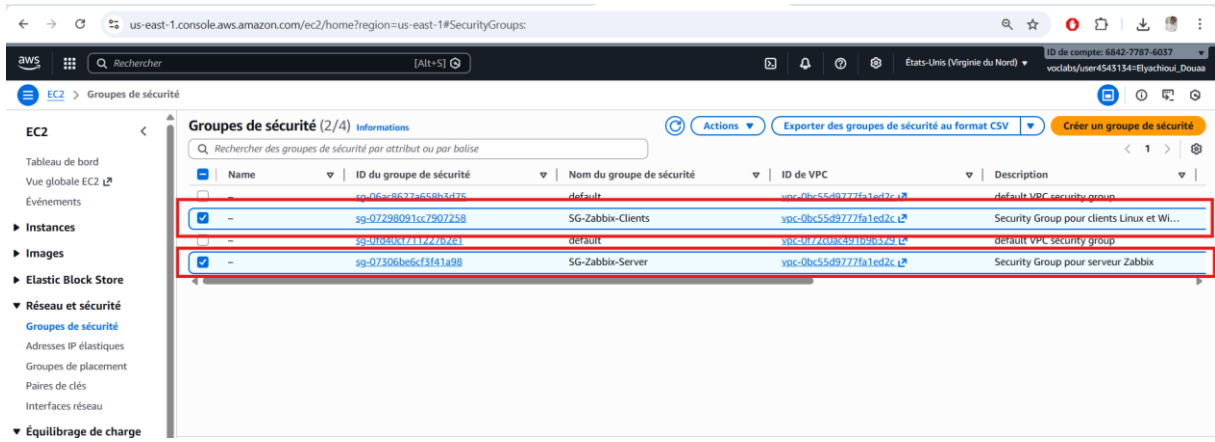
Règles entrantes (3) Gérer les balises Modifier les règles entrantes

☐	Name	ID de règle de grou...	Version IP	Type	Protocole	Plage de ports	Source
☐	-	sg-0f009a577366da4b7	IPv4	RDP	TCP	3389	41.251.155.251/32
☐	-	sg-058fcd5f7411848e	IPv4	SSH	TCP	22	41.251.155.251/32
☐	-	sg-0fa7ebbf5cb93f733	IPv4	TCP personnalisé	TCP	10050	0.0.0.0/0

La création des deux groupes de sécurité, Zabbix-Server-SG pour le serveur et Zabbix-Clients-SG pour les machines clientes, permet d'assurer une sécurité optimale tout en maintenant la communication nécessaire entre le serveur Zabbix et ses clients.

Ces configurations garantissent :

- L'accès sécurisé au serveur (SSH, HTTP/HTTPS, port Zabbix) ;
- L'accès sécurisé aux clients (SSH pour Linux, RDP pour Windows, port agent Zabbix) ;
- La limitation des flux réseau aux seules communications indispensables pour le monitoring ;
- La préparation de l'infrastructure pour le déploiement des instances EC2 et des agents Zabbix.



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#SecurityGroups:

EC2 > Groupes de sécurité

Groupes de sécurité (2/4) Informations

Rechercher des groupes de sécurité par attribut ou par balise

	Name	ID du groupe de sécurité	Nom du groupe de sécurité	ID de VPC	Description
☒	-	sg-06a86377a658b3d25	default	vpc-0bc55d9777fa1ed2c	default VPC security group
☒	-	sg-07298091cc7907258	SG-Zabbix-Clients	vpc-0bc55d9777fa1ed2c	Security Group pour clients Linux et Wi...
☒	-	sg-0f09b0c711227b261	default	vpc-0f72008c391096329	default VPC security group
☒	-	sg-07306b6cf3f1a98	SG-Zabbix-Server	vpc-0bc55d9777fa1ed2c	Security Group pour serveur Zabbix

Figure 25:Création des deux groupes de sécurité, Zabbix-Server-SG pour le serveur et Zabbix-Clients-SG

Partie 2

Création des instances EC2 (Zabbix, Linux, Windows)

Lancement des Instances EC2

Après avoir configuré l'architecture réseau et les groupes de sécurité, l'étape suivante consiste à déployer les instances EC2 qui hébergeront le serveur Zabbix et les clients Linux et Windows. Cette étape est cruciale car elle constitue la base du déploiement du monitoring.

Étape 1 : Accéder au service EC2

1. Se connecter à la **console AWS** via le Learner Lab.
2. Accéder au service **EC2** dans la région **us-east-1 (N. Virginia)**, recommandée pour les Labs.
3. Cliquer sur **Lancer l'instance** pour démarrer la création d'une nouvelle instance.

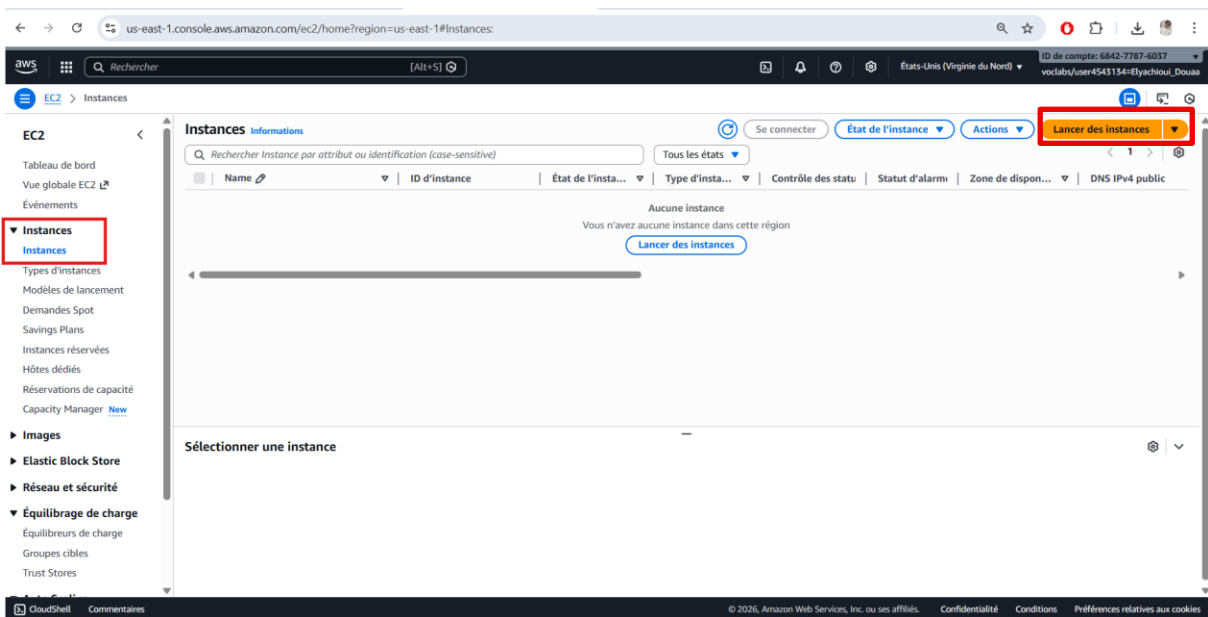


Figure 26: Lancer l'instance pour démarrer la création d'une nouvelle instance

Étape 2 : Lancer le serveur Zabbix

Pour déployer le serveur Zabbix, nous avons créé une instance EC2 dédiée. Les étapes réalisées sont les suivantes :

1. Nom de l'instance

L'instance a été nommée **Douaa-Zabbix-Server** afin de pouvoir l'identifier facilement dans la console AWS. Ce nom est visible dans la liste des instances et dans toutes les configurations ultérieures.

2. Choix de l'AMI

L'image système choisie est **Ubuntu Server 22.04 LTS**. Cette version est recommandée pour le déploiement de Docker et de Zabbix, grâce à sa stabilité, sa compatibilité avec les dernières versions des conteneurs, et sa large documentation disponible.

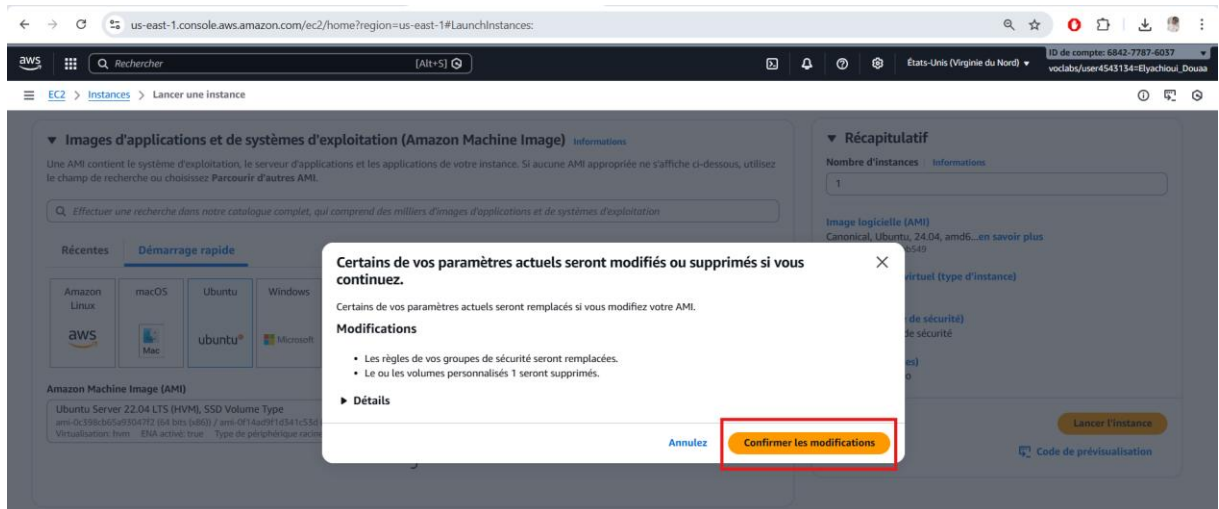


Figure 27: Choix de l'AMI

3. Type d'instance

Le type sélectionné est **t3.medium**, qui fournit **2 CPU** et **4 Go de RAM**. Ce choix respecte les limitations du Learner Lab et permet d'exécuter les conteneurs Docker pour Zabbix Server et l'interface Web avec une performance suffisante pour le laboratoire.

4. Sous-réseau

L'instance a été placée dans le **sous-réseau public Douaa-Zabbix-Public-Subnet**, ce qui lui permet d'obtenir automatiquement une adresse IP publique. Cela garantit l'accès depuis Internet pour l'administration et l'utilisation de l'interface Web de Zabbix.

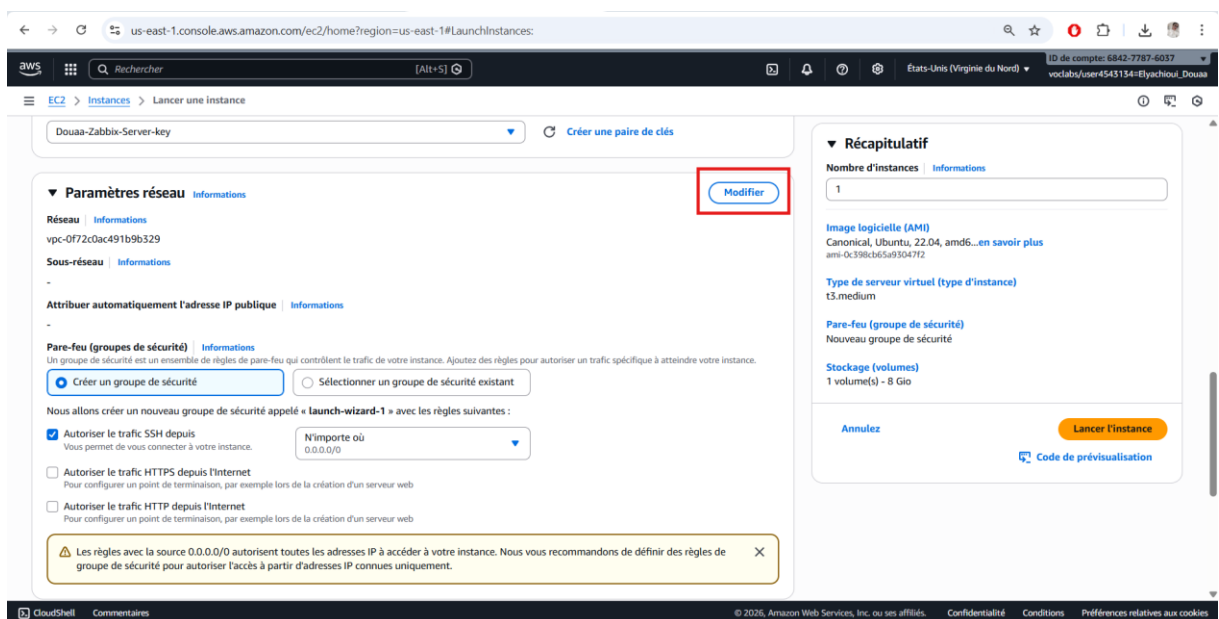


Figure 28: Modification de réseau

▼ Paramètres réseau Informations

VPC - obligatoire Informations

vpc-0bc55d9777fa1ed2c (Douaa-Zabbix-VPC-vpc)
10.0.0.0/16

Sous-réseau Informations

subnet-07539ddde04e0c230 Douaa-Zabbix-Subnet-Public
VPC: vpc-0bc55d9777fa1ed2c Propriétaire: 684277876037 Zone de disponibilité: us-east-1a (use1-az2)
Type de zone: Zone de disponibilité Adresses IP disponibles: 251 CIDR: 10.0.1.0/24

Attribuer automatiquement l'adresse IP publique Informations

Activer

Pare-feu (groupes de sécurité) Informations

Un groupe de sécurité est un ensemble de règles de pare-feu qui contrôlent le trafic de votre instance. Ajoutez des règles pour autoriser un trafic spécifique à atteindre votre instance.

☐ Créer un groupe de sécurité ☒ Sélectionner un groupe de sécurité existant

Groupes de sécurité courants Informations

Sélectionner les groupes de sécurité

SG-Zabbix-Server sg-07306be6cf3f41a98 X
VPC: vpc-0bc55d9777fa1ed2c

Comparer les règles de groupe de sécurité

Les groupes de sécurité que vous ajoutez ou supprimez ici seront ajoutés ou supprimés de toutes vos interfaces réseau.

► Configuration réseau avancée

Figure 29: Configuration des paramètres réseau

5. Clé SSH

Une **paire de clés SSH** a été créée et sélectionnée pour se connecter à l'instance de manière sécurisée. La **clé privée** a été téléchargée et conservée, et elle sera utilisée pour se connecter via un terminal avec la commande :

```
ssh -i "Douaa-Zabbix-Key.pem"
ubuntu@<IP-PUBLIQUE>
```

Cette étape est indispensable pour sécuriser l'accès administratif au serveur.

6. Groupe de sécurité

Le serveur a été associé au groupe de sécurité **Zabbix-Server-SG**, créé précédemment. Ce groupe autorise uniquement les flux nécessaires :

- SSH (port 22) pour l'administration distante,
- HTTP/HTTPS (ports 80 et 443) pour l'interface Web,
- Custom TCP (port 10051) pour recevoir les données des agents Zabbix.

7. Stockage

Le disque par défaut de **8 Go SSD** a été conservé, suffisant pour l'installation de Docker, Zabbix et la base de données.

Créer une paire de clés X

Nom de la paire de clés

Les paires de clés vous permettent de vous connecter à votre instance en toute sécurité.

Douaa-Zabbix-Server-key

La longueur maximale du nom est de 255 caractères ASCII. Il ne peut pas inclure d'espaces avant ou après.

Type de paire de clés

☒ RSA
Paire de clés privée et publique chiffrée RSA

☐ ED25519
Paire de clés privée et publique chiffrée ED25519

Format de fichier de clé privée

☒ .pem
À utiliser avec OpenSSH

☐ .ppk
À utiliser avec PuTTY

Lorsque vous y êtes invité, stockez la paire de clés dans un emplacement sécurisé et accessible sur votre ordinateur. Vous en aurez besoin ultérieurement pour vous connecter à votre instance. En savoir plus

Annulez Créer une paire de clés

Figure 30: Clé SSH

8. Lancement de l'instance

Après avoir configuré tous les paramètres, nous avons cliqué sur **Launch Instance**. L'instance est apparue dans le tableau de bord EC2 avec le statut **“Running”**, indiquant qu'elle est prête pour la configuration de Zabbix.

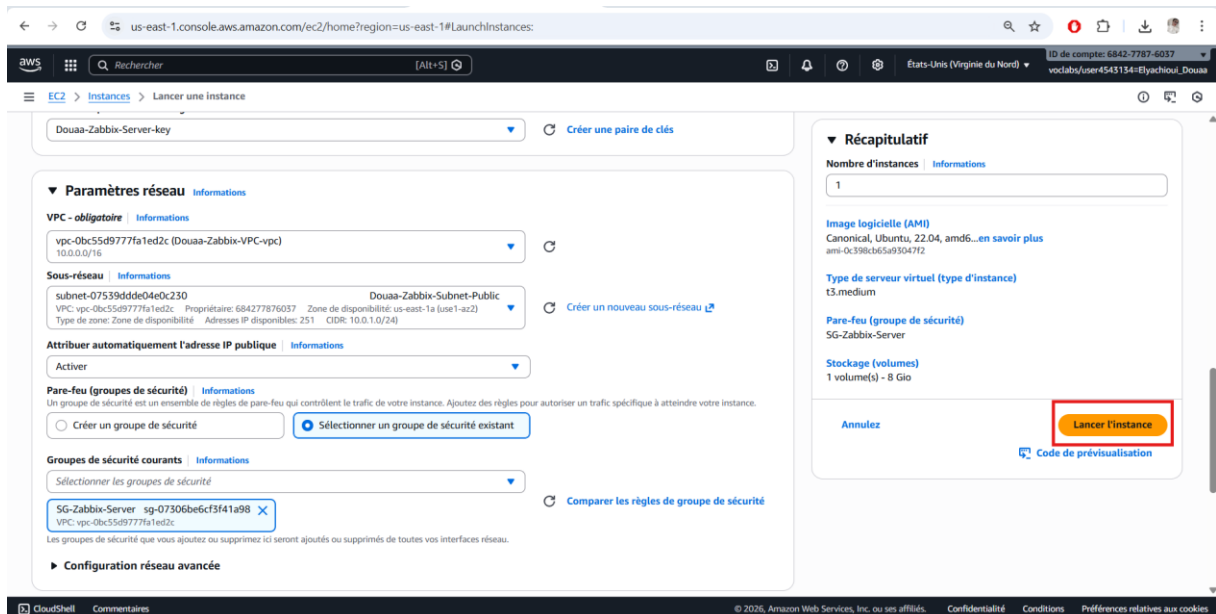


Figure 31: Lancement de l'instance

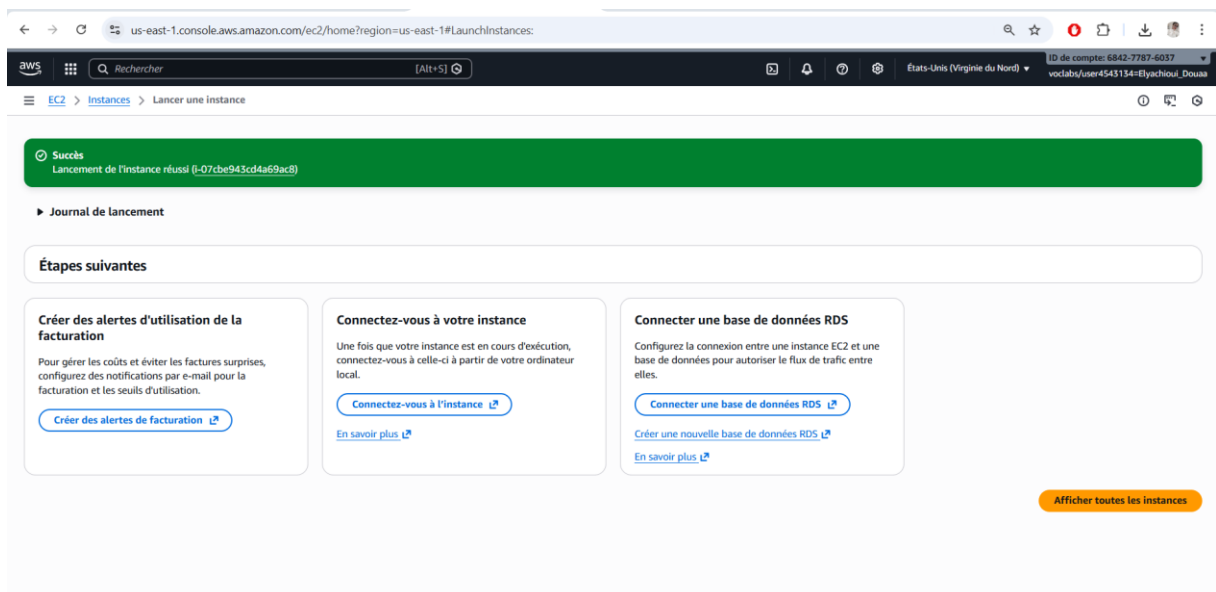


Figure 32: Création réussie de l'instance EC2 du serveur Zabbix

Étape 3 : Lancement de l'instance Linux

L'instance **Douaa-Zabbix-Linux** a été déployée en suivant la même procédure que pour le serveur Zabbix. Nous avons utilisé **Ubuntu Server 22.04 LTS** comme système d'exploitation et le type **t3.micro** avec **8 Go de RAM**, suffisant pour exécuter l'agent Zabbix et assurer la surveillance du serveur dans le contexte du Learner Lab.

L'instance a été placée dans le **sous-réseau public Douaa-Zabbix-Public-Subnet** pour obtenir automatiquement une adresse IP publique, et associée au **groupe de sécurité des clients Zabbix (Zabbix-Clients-SG)**, autorisant SSH, le port de l'agent Zabbix et les tests de connectivité.

Une **clé SSH** a été utilisée pour accéder à l'instance depuis le terminal de manière sécurisée. Cette configuration assure que le client Linux peut communiquer correctement avec le serveur Zabbix et envoyer ses métriques de monitoring.

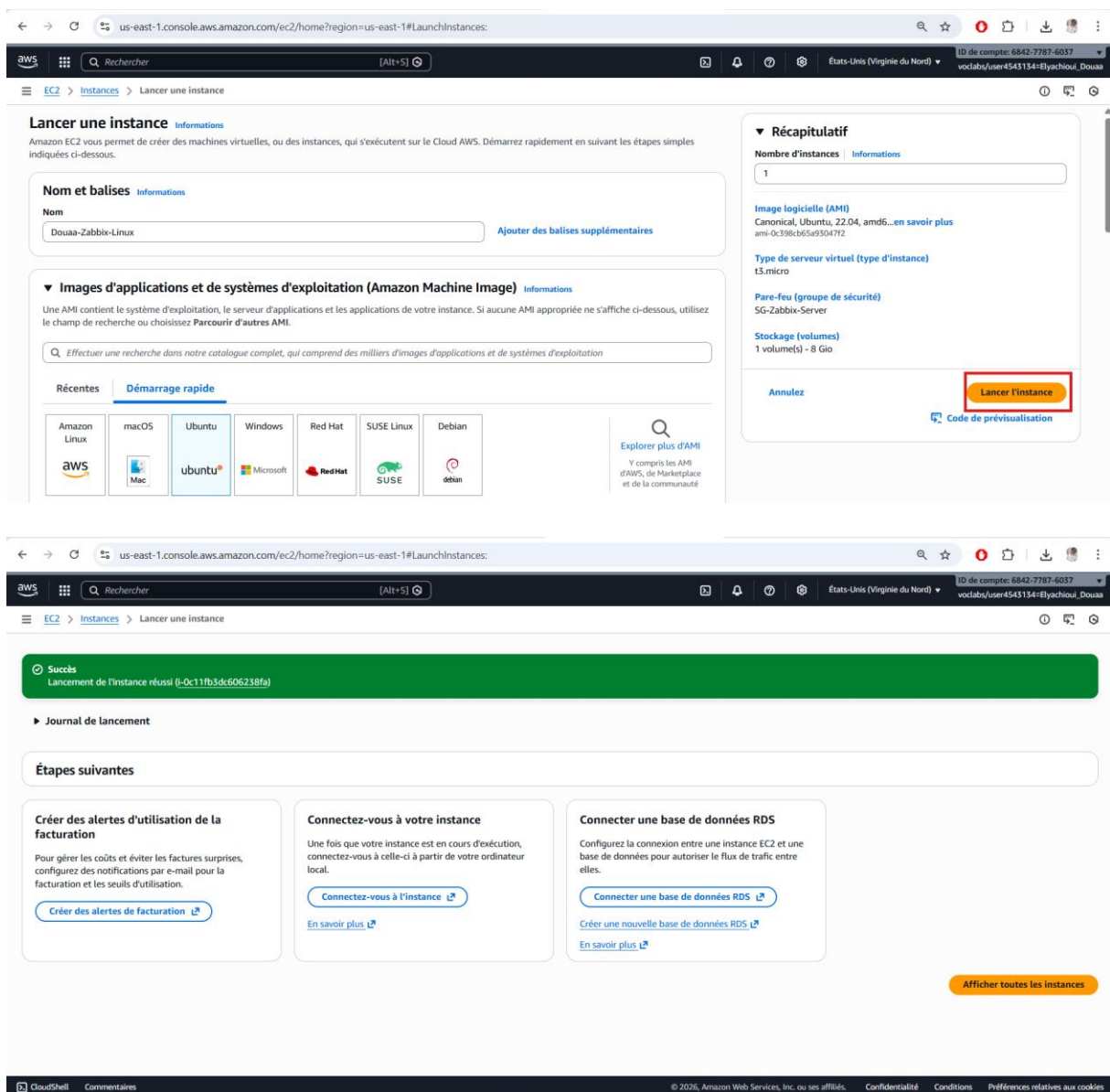


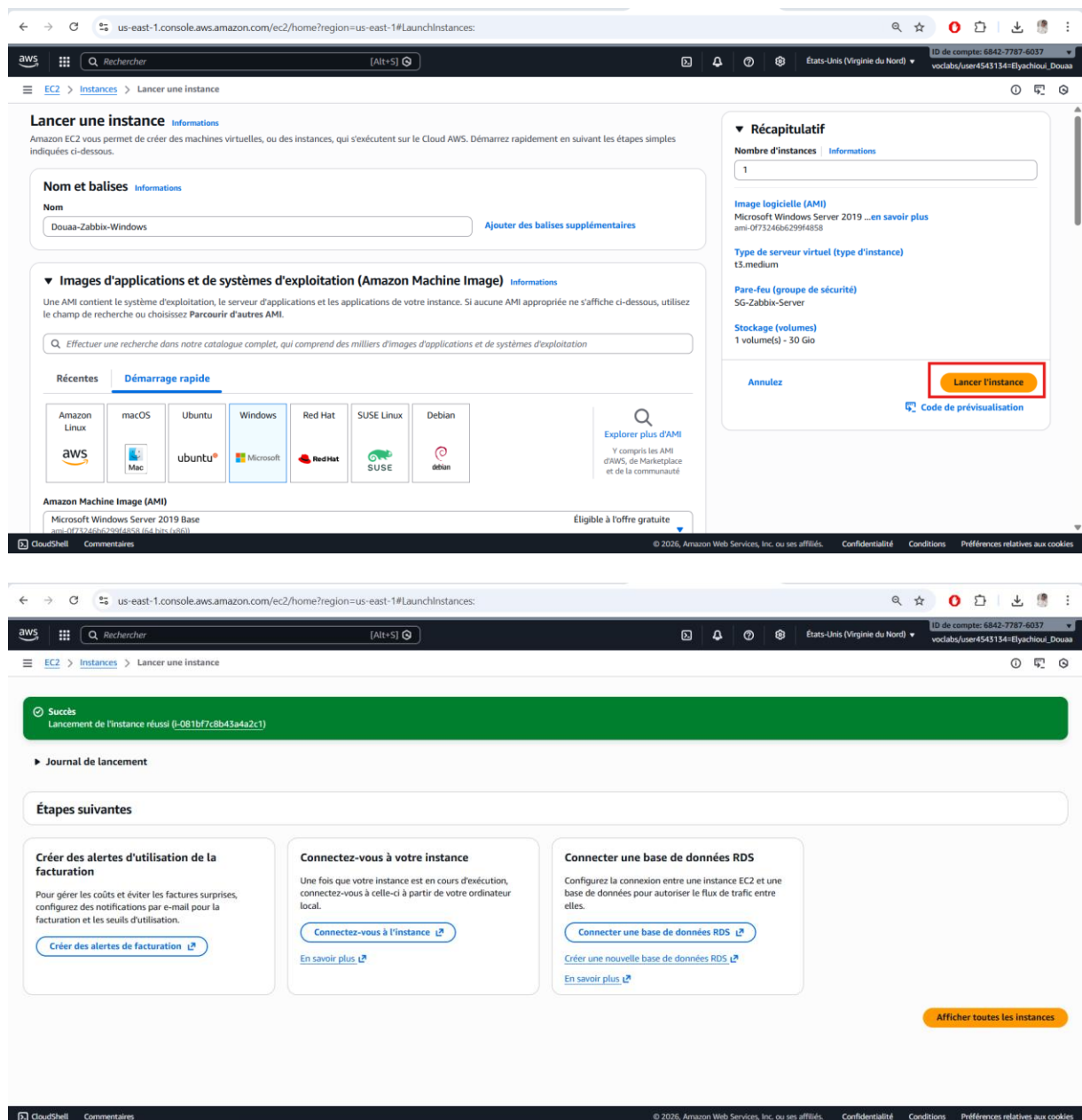
Figure 33: Succès du lancement de l'instance Linux

Étape 4 : Lancement de l'instance Windows

L'instance **Douaa-Zabbix-Windows** a été déployée en suivant la même procédure que les autres instances. Nous avons utilisé **Windows Server 2019** comme système d'exploitation et le type **t3.medium** avec **30 Go** de stockage, suffisant pour exécuter l'agent Zabbix et assurer la surveillance via l'interface du serveur.

L'instance a été placée dans le sous-réseau public **Douaa-Zabbix-Public-Subnet** pour obtenir automatiquement une adresse IP publique, et associée au groupe de sécurité des clients Zabbix (**Zabbix-Clients-SG**), permettant :

- RDP (port 3389) pour accéder à l'interface graphique du serveur Windows.
- Custom TCP (port 10050) pour que l'agent Zabbix envoie ses données au serveur central.



The screenshot displays the AWS Management Console interface for launching an EC2 instance. The top navigation bar shows the user is logged in as 'voclabr/user4543134-Elyachoui_Douaa' in the 'us-east-1' region.

Top Section: Lancer une instance

- Nom et balises:** The instance name is 'Douaa-Zabbix-Windows'.
- Images d'applications et de systèmes d'exploitation (Amazon Machine Image):** The 'Windows' tab is selected, showing the 'Microsoft Windows Server 2019 Base' AMI.
- Récapitulatif:**
 - Nombre d'instances: 1
 - Image logicielle (AMI): Microsoft Windows Server 2019 ...
 - Type de serveur virtuel (type d'instance): t3.medium
 - Par-feu (groupe de sécurité): SG-Zabbix-Server
 - Stockage (volumes): 1 volume(s) - 30 Gio
 - A 'Lancer l'instance' button is highlighted with a red box.

Bottom Section: Succès

A green banner indicates 'Succès' (Success) with the message 'Lancement de l'instance réussi (i-081bf7c8b43a4a2c1)'. Below this, the 'Étapes suivantes' (Next steps) section provides instructions on how to connect to the instance, create alerts, and connect to a database.

Figure 34: Succès du lancement de l'instance Windows

Les trois instances EC2 déployées le serveur Zabbix, le client Linux et le client Windows sont désormais opérationnelles et correctement configurées avec leurs groupes de sécurité respectifs et adresses IP publiques, prêtes pour le déploiement et la supervision du parc hybride.

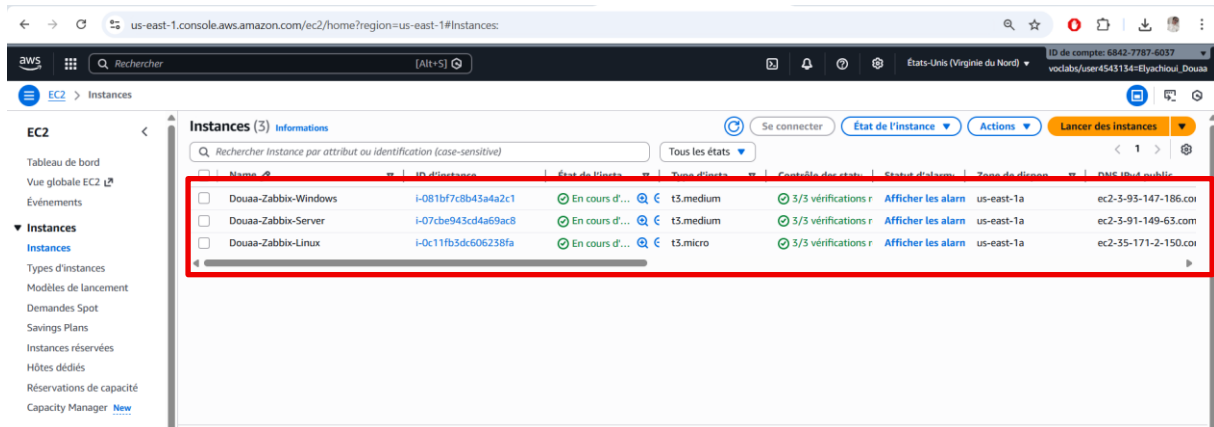


Figure 35: Les trois instances EC2 déployées

Partie 3

Déploiement du Serveur Zabbix

Étape 1 : Connexion au serveur Zabbix

Pour commencer le déploiement, nous devons nous connecter à l'instance **Douaa-Zabbix-Server** via SSH. Cette connexion permet d'exécuter les commandes nécessaires pour installer Docker, Docker-Compose et les conteneurs Zabbix.

Commande utilisée :

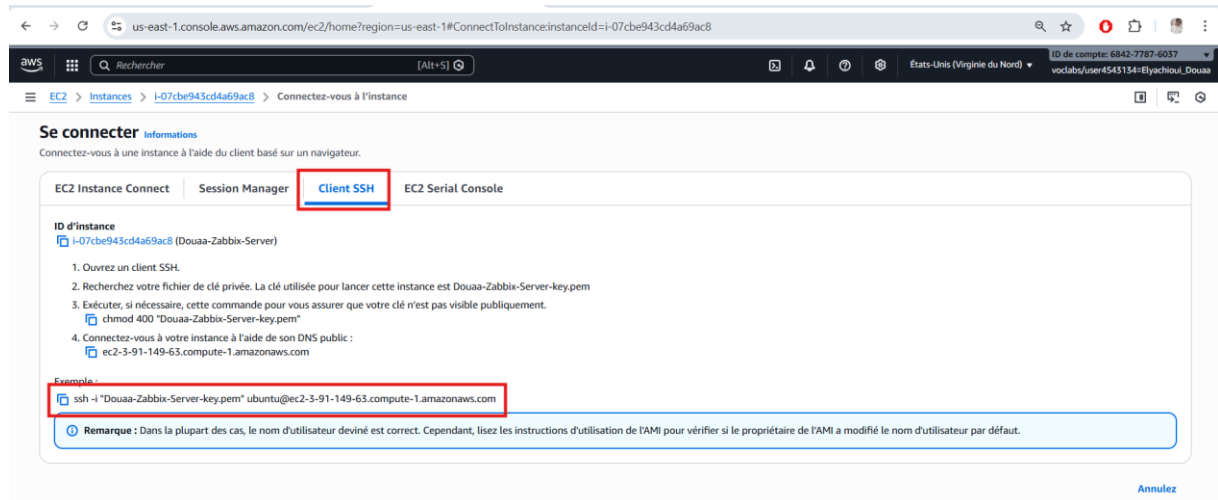


Figure 36: Client SSH

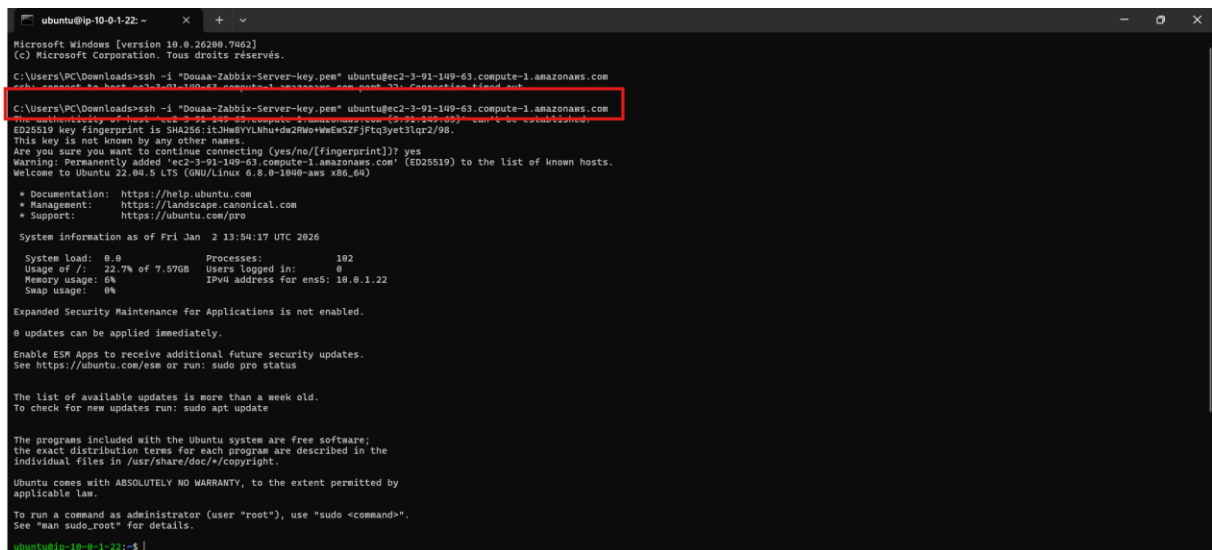


Figure 37: Connection à l'instance Douaa-Zabbix-Server via SSH

Étape 2 : Installation de Docker

– Mise à jour des paquets

Avant toute installation, nous avons mis à jour la liste des paquets disponibles afin de garantir l'installation des dernières versions :

sudo apt update

Cette commande permet d'actualiser les informations des dépôts et de préparer le serveur à l'installation.

```
ubuntu@ip-10-0-1-22: ~$ sudo apt update
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-10-0-1-22:~$ sudo apt update
sudo apt upgrade -y
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [14.1 MB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe Translation-en [5652 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 c-n-f Metadata [286 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages [217 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse Translation-en [112 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse amd64 c-n-f Metadata [8372 B]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3161 kB]
Get:12 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [484 kB]
Get:13 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.0 kB]
Get:14 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [5043 kB]
Get:15 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [944 kB]
Get:16 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 c-n-f Metadata [644 B]
Get:17 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1244 kB]
Get:18 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [319 kB]
Get:19 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 c-n-f Metadata [30.0 kB]
Get:20 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 Packages [57.6 kB]
Get:21 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/multiverse Translation-en [13.2 kB]
Get:22 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/main amd64 Packages [690 B]
Get:23 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/main amd64 c-n-f Metadata [69.4 kB]
Get:24 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/main Translation-en [11.5 kB]
Get:25 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/main amd64 c-n-f Metadata [412 B]
Get:26 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/restricted amd64 c-n-f Metadata [116 B]
Get:27 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/universe amd64 Packages [31.7 kB]
Get:28 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/universe Translation-en [16.9 kB]
```

Figure 38: Mise à jour des paquets

– Installation de Docker

Docker a été installé avec la commande suivante :

sudo apt install docker.io -y

- L'option -y permet de valider automatiquement l'installation.
- Après installation, Docker est prêt à être utilisé pour gérer les conteneurs.

```
ubuntu@ip-10-0-1-22: ~$ sudo apt install docker.io -y
Scanning processes...
Scanning candidates...
Scanning linux images...

Restarting services...
/etc/needrestart/restart.d/systemd-manager
systemctl restart packagekit.service
Service restarts being deferred:
/etc/needrestart/restart.d/dbus.service
systemctl restart networkd-dispatcher.service
systemctl restart unattended-upgrades.service
systemctl restart user@1000.service

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-10-0-1-22:~$ sudo apt install docker.io -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools cgroupfs-mount | cgroup-lite debotstrap docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils
The following NEW packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base docker.io pigz runc ubuntu-fan
0 upgraded, 8 newly installed, 0 to remove and 0 not upgraded.
Need to get 76.3 MB of archives.
After this operation, 289 MB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 pigz amd64 2.6-1 [63.6 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/main amd64 bridge-utils amd64 1.7-1ubuntu3 [34.4 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 runc amd64 1.3.3-0ubuntu1-22.04.3 [8857 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 containerd amd64 1.7.28-0ubuntu1-22.04.1 [38.5 MB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 dns-root-data all 2024071801-ubuntu0.22.04.1 [6132 B]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 dnsmasq-base amd64 2.90-0ubuntu0.22.04.1 [374 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 docker.io amd64 28.2.2-0ubuntu1-22.04.1 [28.4 MB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 ubuntu-fan all 0.12.16 [35.2 kB]
Fetched 76.3 MB in 1s (94.2 MB/s)
Preconfiguring packages ...
Selecting previously unselected package pigz.
(Reading database ... 96599 files and directories currently installed.)
Preparing to unpack .../0-pigz_2.6-1_amd64.deb ...
Unpacking pigz (2.6-1) ...
```

Figure 39: Installation de Docker

– Activation et démarrage du service Docker

Pour que Docker fonctionne correctement et démarre automatiquement à chaque redémarrage du serveur, nous avons exécuté :

sudo systemctl start docker
sudo systemctl enable docker

```
ubuntu@ip-10-0-1-22:~$ sudo systemctl start docker
ubuntu@ip-10-0-1-22:~$ sudo systemctl enable docker
```

Figure 40: Activation et démarrage du service Docker

- start : lance immédiatement le service Docker
- enable : active Docker au démarrage du serveur

– Vérification de l'installation

Enfin, nous avons vérifié la version installée pour confirmer que Docker est correctement installé :

docker --version

```
ubuntu@ip-10-0-1-22:~$ sudo docker --version
Docker version 20.10.2, build 20.10.2~ubuntu-22.04.1
ubuntu@ip-10-0-1-22:~$
```

Figure 41: Vérification de l'installation

La commande affiche la version de Docker, assurant que l'installation a été réalisée avec succès.

Étape 3 : Installer Docker-Compose

Après l'installation de Docker, il est nécessaire d'installer **Docker-Compose**, qui permet de gérer plusieurs conteneurs simultanément et de lancer les services Zabbix (serveur, interface Web et base de données) de manière coordonnée.

1. Installation de Docker-Compose

Sur le serveur, nous avons installé Docker-Compose avec la commande suivante :

sudo apt install docker-compose -y

- L'option -y permet de valider automatiquement l'installation.

```

ubuntu@ip-10-0-1-22:~$ sudo systemctl start docker
ubuntu@ip-10-0-1-22:~$ sudo systemctl enable docker
ubuntu@ip-10-0-1-22:~$ sudo docker --version
Docker version 20.10.7, build 20.10.7-ubuntu~1.22.00.1
ubuntu@ip-10-0-1-22:~$ sudo apt install docker-compose -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-docker python3-dockerpty python3-docopt python3-dotenv python3-texttable python3-websocket
The following NEW packages will be installed:
  docker-compose python3-docker python3-dockerpty python3-docopt python3-dotenv python3-texttable python3-websocket
0 upgraded, 7 newly installed, 0 to remove and 0 not upgraded.
Need to get 290 kB of archives.
After this operation, 1545 kB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-websocket all 1.2.3-1 [34.7 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-docker all 5.0.3-1 [89.3 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-dockerpty all 0.4.1-2 [11.1 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-docopt all 0.6.2-4 [26.9 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-dotenv all 0.19.2-1 [20.5 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-texttable all 1.6.4-1 [11.4 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 docker-compose all 1.29.2-1 [95.8 kB]
Fetched 290 kB in 0s (8772 kB/s)

```

Figure 42: Installation de Docker-Compose

Cette commande installe la version disponible dans les dépôts Ubuntu, suffisante pour le laboratoire.

2. Vérification de l'installation

Après l'installation, nous avons vérifié que Docker-Compose fonctionne correctement en affichant sa version :

docker-compose --version

```

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-10-0-1-22:~$ docker-compose --version
docker-compose version 1.29.2, build unknown
ubuntu@ip-10-0-1-22:~$

```

Figure 43: Vérification de l'installation

Cette étape est importante pour confirmer que Docker-Compose est bien installé et prêt à être utilisé pour déployer les conteneurs Zabbix.

Étape 4 : Lancer Zabbix avec Docker

Pour déployer Zabbix sur le serveur, nous avons utilisé **Docker-Compose**, qui permet de gérer simultanément plusieurs conteneurs pour le serveur Zabbix, l'interface Web et la base de données.

1. Création d'un dossier pour Zabbix

Nous avons commencé par créer un répertoire dédié à Zabbix afin d'organiser les fichiers de configuration :

```
mkdir ~/zabbix-docker  
cd ~/zabbix-docker
```

- Ce dossier contiendra le fichier docker-compose.yml ainsi que tous les fichiers nécessaires au déploiement.

2. Création du fichier docker-compose.yml

Le fichier docker-compose.yml définit les services Docker à lancer : serveur Zabbix, base de données et interface Web.

Nous avons créé le fichier avec la commande : **nano docker-compose.yml**



```
GNU nano 6.2 docker-compose.yml *
version: '3.5'

services:
  zabbix-server:
    image: zabbix/zabbix-server-mysql:latest
    container_name: zabbix-server
    environment:
      - DB_SERVER_HOST=zabbix-db
      - MYSQL_USER=zabbix
      - MYSQL_PASSWORD=zabbix
    ports:
      - "10051:10051"
    depends_on:
      - zabbix-db

  zabbix-web:
    image: zabbix/zabbix-web-apache-mysql:latest
    container_name: zabbix-web
    environment:
      - DB_SERVER_HOST=zabbix-db
      - MYSQL_USER=zabbix
      - MYSQL_PASSWORD=zabbix
      - ZBX_SERVER_HOST=zabbix-server
    ports:
      - "80:8080"
    depends_on:
      - zabbix-server

  zabbix-db:
    image: mysql:5.7
    container_name: zabbix-db
    environment:
      - MYSQL_ROOT_PASSWORD=root
      - MYSQL_DATABASE=zabbix
      - MYSQL_USER=zabbix
      - MYSQL_PASSWORD=zabbix
    volumes:
```

Figure 44:Création du fichier docker-compose.yml

- À l'intérieur, nous avons défini les services avec les paramètres suivants :
 - **Zabbix Server** : port 10051
 - **Web Interface** : ports 80/443
 - **Base de données** : MySQL ou PostgreSQL selon le choix
 - **Volumes** : pour conserver les données même après l'arrêt des conteneurs

- Chaque service a été configuré avec ses variables d'environnement nécessaires pour fonctionner correctement.

3. Lancement des conteneurs Zabbix

Une fois le fichier prêt, nous avons lancé les conteneurs en arrière-plan avec la commande :

docker-compose up -d

```

ubuntu@ip-10-0-1-22: ~/zabbix$ sudo docker-compose up -d
Creating network 'zabbix_default' with the default driver
Pulling zabbix-db (mysql:5.7)...
5.7: Pulling from library/mysql
28e4dca8c69: Pull complete
1c56c3d4ce74: Pull complete
e9f03a1c24ce: Pull complete
68c398c2015: Pull complete
6b95a948e766: Pull complete
90986bb8de6e: Pull complete
ae71319cb779: Pull complete
4fc89e9d4d8: Pull complete
43d85e938198: Pull complete
864b2d298fba: Pull complete
df9a4d85569b: Pull complete
Digest: sha256:48c0b0c36ed8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb
Status: Downloaded newer image for mysql:5.7
Pulling zabbix-server (zabbix/zabbix-server-mysql:latest)...
latest: Pulling from zabbix/zabbix-server-mysql
1974333eccd: Pull complete
85123c57b00f: Pull complete
ecc8051bf8fa: Pull complete
38ea57a43c3a: Pull complete
3806a2c0aa3: Pull complete
699eeef19a3ef: Pull complete
4f4fb700ef54: Pull complete
46a41216a95: Pull complete
Digest: sha256:c67191ea38272704b12ee7d17a183d4b6b728b12c6f7f153efcac6f1e21ad6f
Status: Downloaded newer image for zabbix/zabbix-server-mysql:latest
Pulling zabbix-web (zabbix/zabbix-web-apache-mysql:latest)...
latest: Pulling from zabbix/zabbix-web-apache-mysql
1974333eccd: Already exists
857121b1b697: Pull complete
8819c7408492: Pull complete
74b09832d645: Pull complete
4f4fb700ef54: Pull complete
bdf956e9fe94: Pull complete
Digest: sha256:880e86c19e81ed65e788bc46876b6c5e5f4a75975ba5a86d7f7696293a18beb
Status: Downloaded newer image for zabbix/zabbix-web-apache-mysql:latest
Creating zabbix-db ... done
Creating zabbix-server ... done
Creating zabbix-web ... done
ubuntu@ip-10-0-1-22: ~/zabbix$
  
```

Figure 45: Lancement des conteneurs Zabbix

- L'option -d permet de lancer les conteneurs **en arrière-plan**, laissant le terminal libre pour d'autres commandes.
- Docker-Compose télécharge automatiquement les images nécessaires et démarre les services définis dans le fichier YAML.

4. Vérification du fonctionnement

Pour s'assurer que tous les conteneurs sont correctement lancés, nous avons utilisé :

docker ps

```

ubuntu@ip-10-0-1-22: ~/zabbix$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED    STATUS      PORTS
19ed04ae42e    zabbix/zabbix-web-apache-mysql:late "docker-entrypoint.sh"   11 seconds ago    Up 11 seconds    8443/tcp, 0.0.0.0:80->8080/tcp, [::]:80->8080/tcp
821c0a08d2e    zabbix/zabbix-server-mysql:latest   "/usr/bin/docker-ent-"  11 seconds ago    Up 11 seconds    0.0.0.0:10051->10051/tcp, [::]:10051->10051/tcp
3114121972ef   mysql:5.7                           "docker-entrypoint s-"  12 seconds ago    Up 11 seconds    3306/tcp, 33060/tcp
  
```

Figure 46: Vérification du fonctionnement

- Cette commande liste tous les conteneurs en cours d'exécution et leur statut.
- Les conteneurs **Zabbix Server**, **Zabbix Web Interface** et **Base de données** doivent apparaître avec le statut **Up**, indiquant que le serveur est prêt à recevoir les clients.

Étape 5 : Accéder à l'interface Web et se connecter à Zabbix

Après avoir lancé les conteneurs avec Docker-Compose, nous avons vérifié que **tous les services étaient opérationnels** (Zabbix Server, Web Interface et Base de données) en utilisant :

docker ps

Tous les conteneurs devaient apparaître avec le statut **Up**, confirmant que le serveur Zabbix est prêt.

1. Accès à l'interface Web

Nous avons ouvert un navigateur et saisi l'adresse publique du serveur Zabbix :

- L'interface Web de Zabbix s'est affichée après quelques minutes, le temps que les conteneurs terminent leur initialisation.
- Cela permet de configurer les clients et de visualiser les métriques de monitoring.

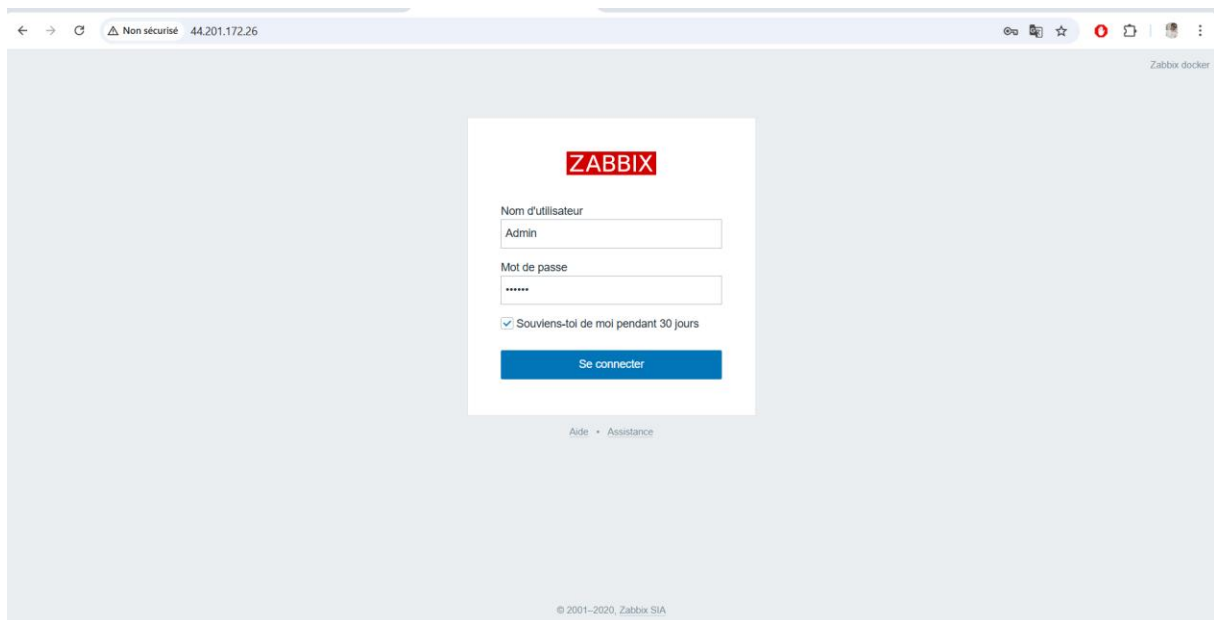


Figure 47: Interface Web de Zabbix affichée dans le navigateur

2. Connexion avec le compte administrateur

Pour se connecter, nous avons utilisé les identifiants par défaut fournis par Zabbix :

- **Login : Admin**
- **Mot de passe : zabbix**

Après la première connexion, il est recommandé de **changer le mot de passe administrateur** pour des raisons de sécurité.

3. Vérification du fonctionnement

Une fois connecté :

- ✓ Le tableau de bord principal affiche l'état du serveur Zabbix et des services associés.

- ✓ Nous avons vérifié que le serveur est **prêt à recevoir les données des agents Linux et Windows**.
- ✓ Cette étape confirme que le serveur est opérationnel et prêt pour le déploiement complet du monitoring du parc hybride.

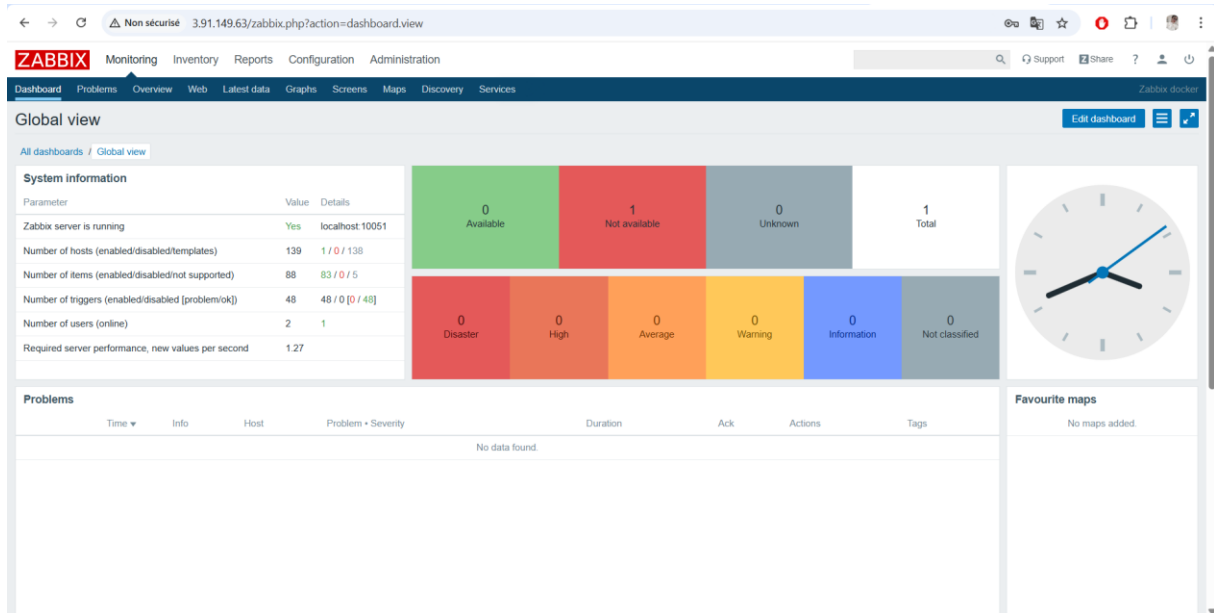


Figure 48: Tableau de bord Zabbix prêt à recevoir les clients

Partie 4

Configuration des Clients (Agents) et Monitoring et Tableaux de Bord

Création des agents :

Les agents Zabbix ont été créés pour les clients Linux et Windows afin de permettre la collecte des métriques système par le serveur.

Les agents ont été configurés **à la fois depuis la ligne de commande** sur chaque machine cliente et **directement depuis l'interface Web de Zabbix**. Cette double approche permet de s'assurer que les clients communiquent correctement avec le serveur.

Client Linux

Le client Linux a été configuré pour envoyer ses métriques au serveur Zabbix.

– Création et configuration des agents :

- Depuis la **ligne de commande**, l'agent a été installé et configuré pour communiquer avec le serveur Zabbix :

1. Installation de l'agent Zabbix

Nous avons installé l'agent Zabbix sur l'instance Linux avec les commandes suivantes :

```
sudo apt update  
sudo apt install zabbix-agent -y
```

- L'agent permet au serveur Zabbix de collecter des informations sur le système (CPU, RAM, disque, etc.).

2. Configuration du fichier zabbix_agentd.conf

Le fichier de configuration principal de l'agent se trouve dans :
/etc/zabbix/zabbix_agentd.conf

- Nous avons modifié les paramètres essentiels :
 - Server= 10.0.1.250
 - Hostname=Client Linux
- Après modification, nous avons redémarré l'agent pour appliquer les changements :

```
sudo systemctl restart zabbix-agent  
sudo systemctl enable zabbix-agent
```

- Depuis l'**interface Web de Zabbix**, nous avons également ajouté et configuré hôte pour confirmer sa présence et sa connectivité.

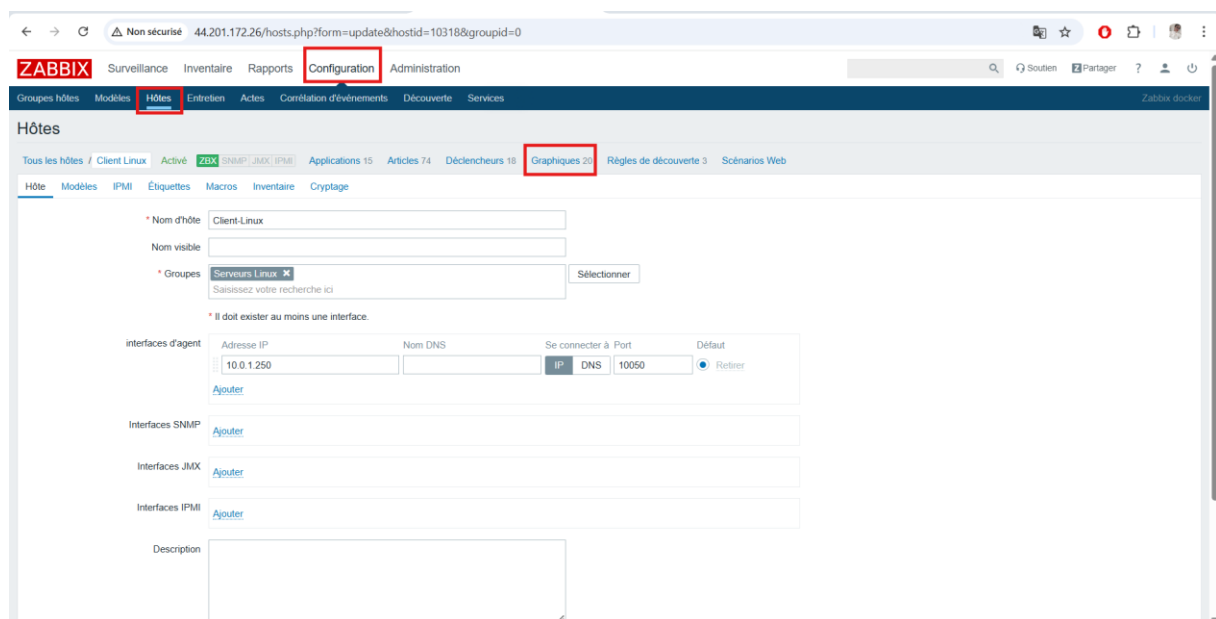
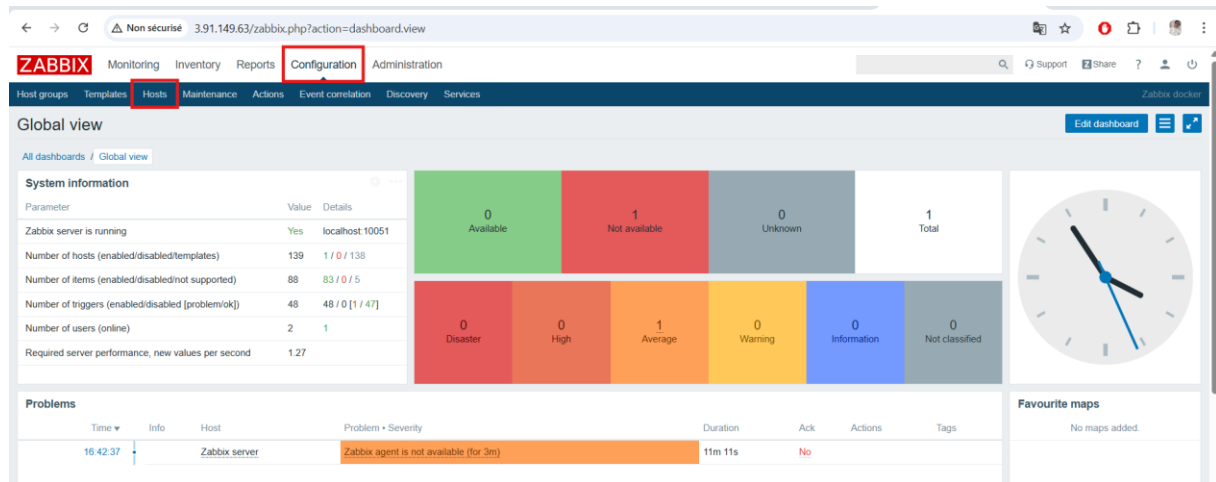


Figure 49: Ajout hôte Linux dans l'interface Zabbix

3. Vérification de la connexion

Nous avons testé que le serveur reçoit bien les données du client Linux via l'interface Web Zabbix.

- Le statut de l'agent doit apparaître **"Vert"** dans l'onglet **Configuration > Hosts**.

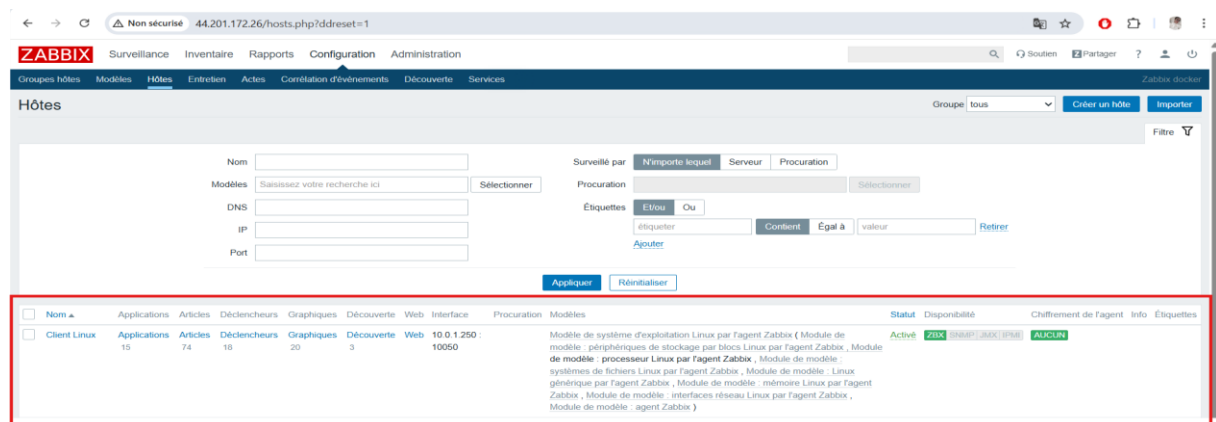
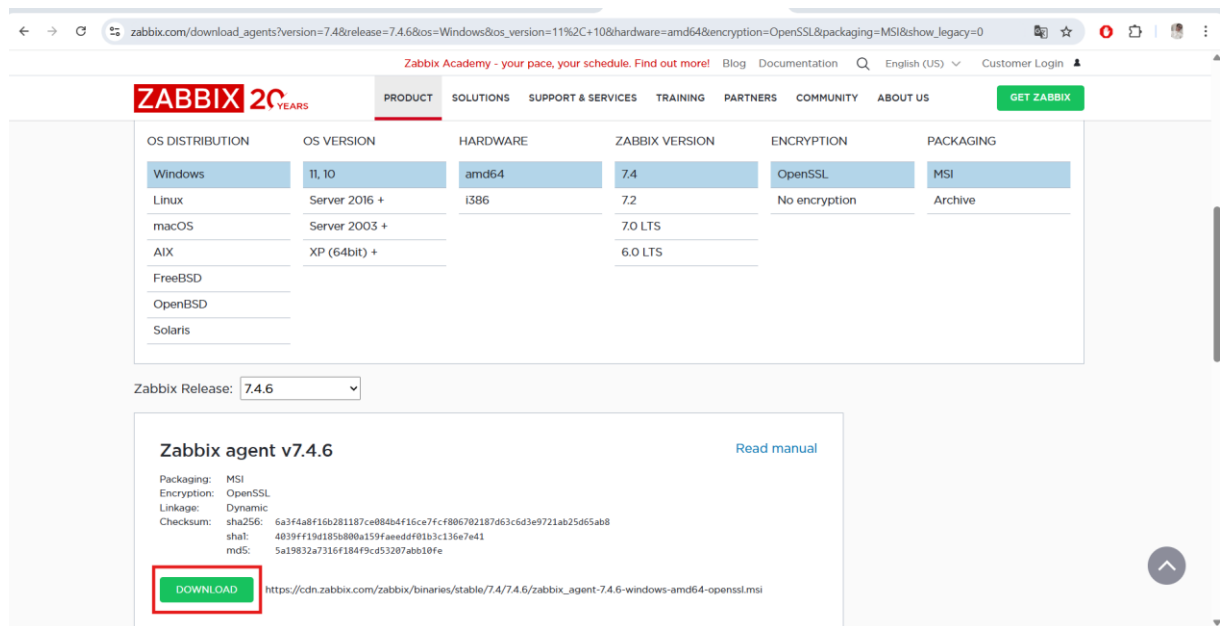


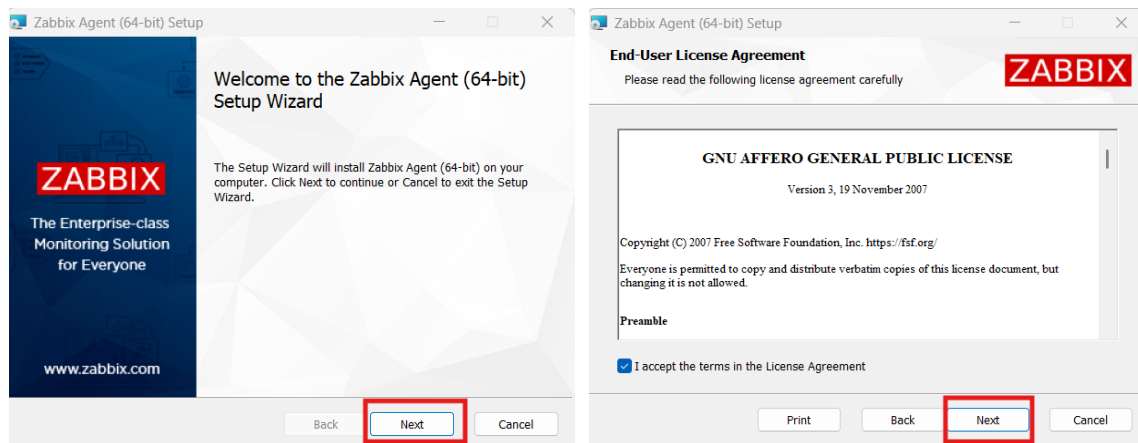
Figure 50: Statut "Vert" (ZBX) de client Linux

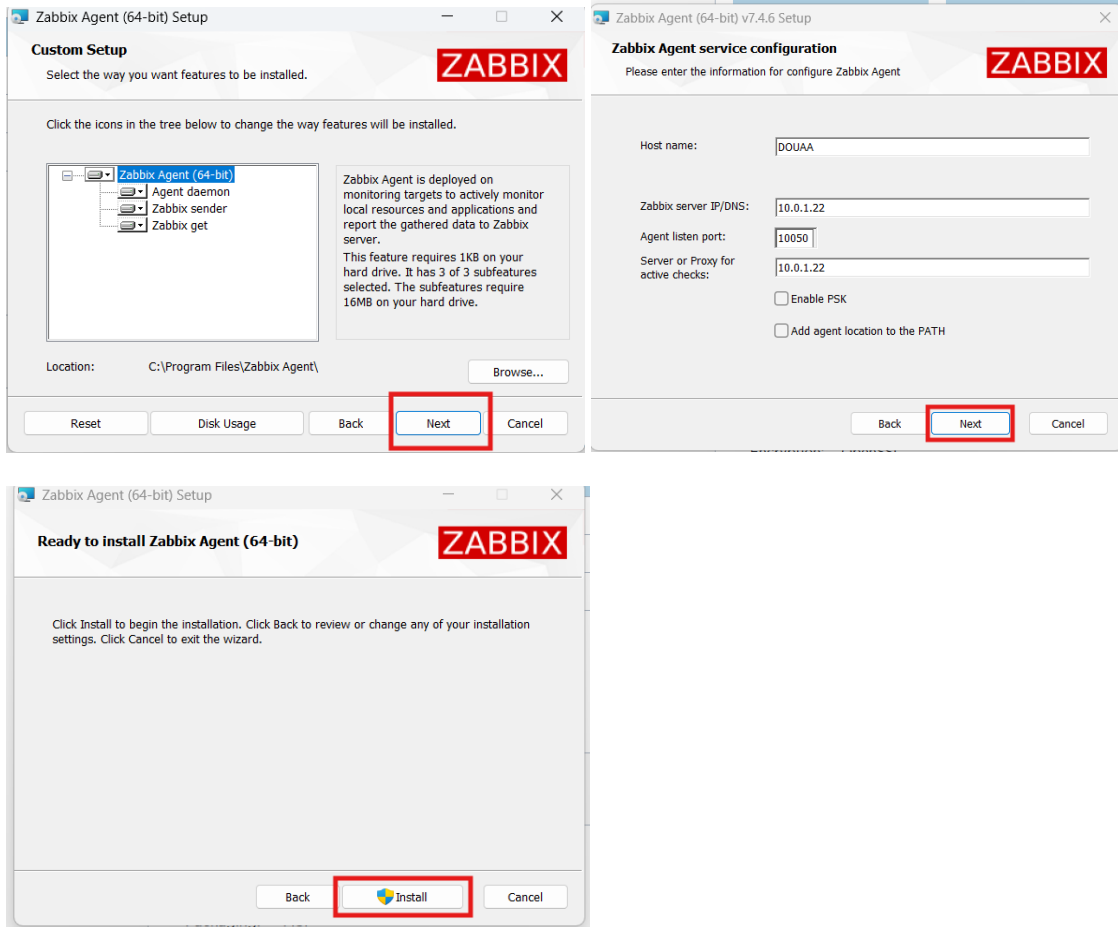
Client Windows

Après avoir installé l'agent Zabbix sur l'instance Windows depuis le **site officiel**, nous avons configuré l'agent pour qu'il puisse communiquer avec le serveur Zabbix.

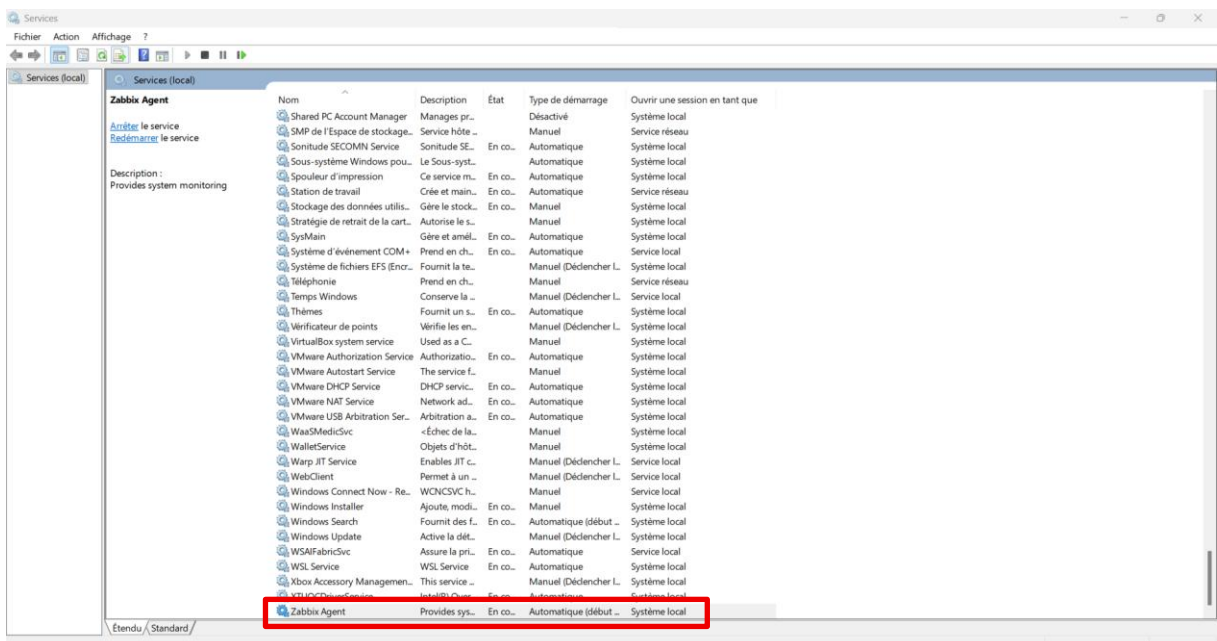


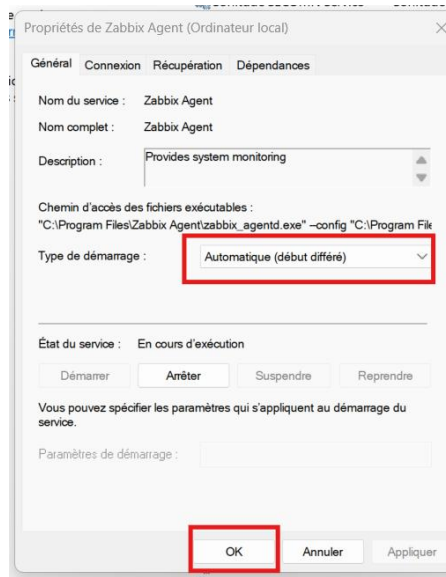
Pour le client Windows, nous avons téléchargé et installé l'agent Zabbix en cliquant sur le fichier **zabbix_agent-7.4.6-windows-amd64-openssl.msi** depuis le site officiel, comme illustré dans les captures d'écran.





Avant de lancer l'agent, nous avons ouvert services.msc via Win + R, localisé le service **Zabbix Agent**, puis configuré son type de démarrage sur **Automatique** afin qu'il se lance automatiquement à chaque démarrage de l'instance.





1. Ouverture du fichier de configuration

Le fichier principal zabbix_agentd.conf se trouve généralement dans :

C:\Program Files\Zabbix Agent\

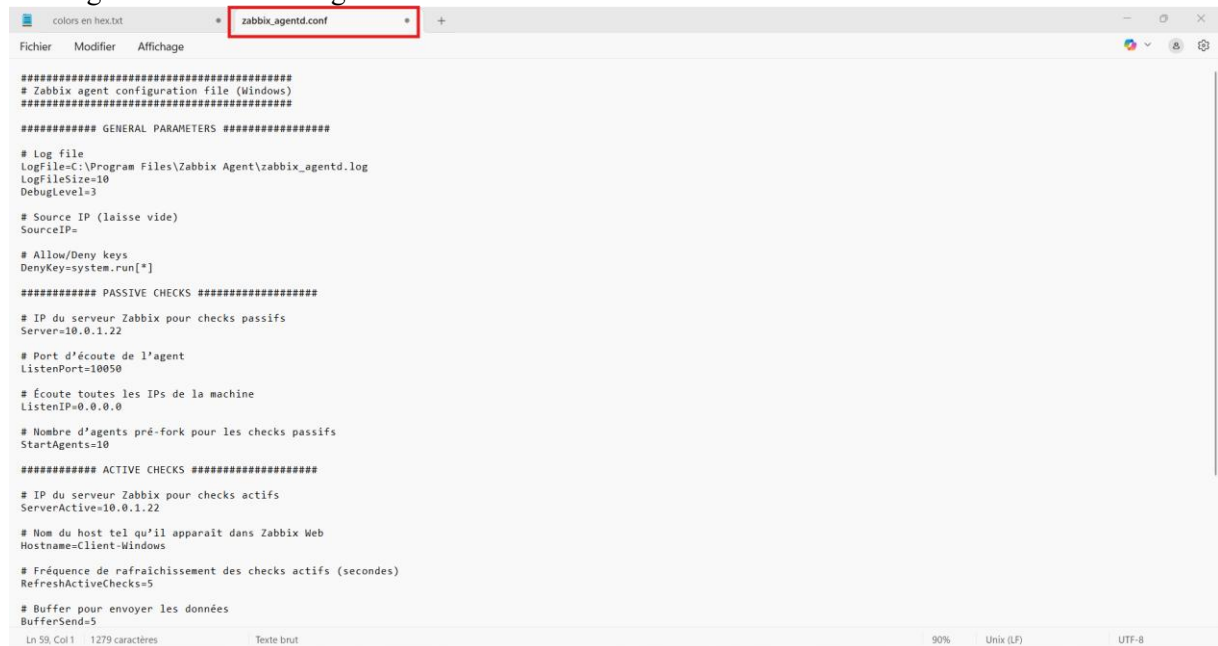


Figure 51: Configuration du fichier zabbix_agentd.conf.

- Server=10.0.1.22
- Hostname= Client Windows

2. Démarrage du service de l'agent via commande

Pour que l'agent fonctionne et démarre automatiquement, nous avons utilisé PowerShell :

Start-Service "Zabbix Agent"
Set-Service "Zabbix Agent" -StartupType Automatic

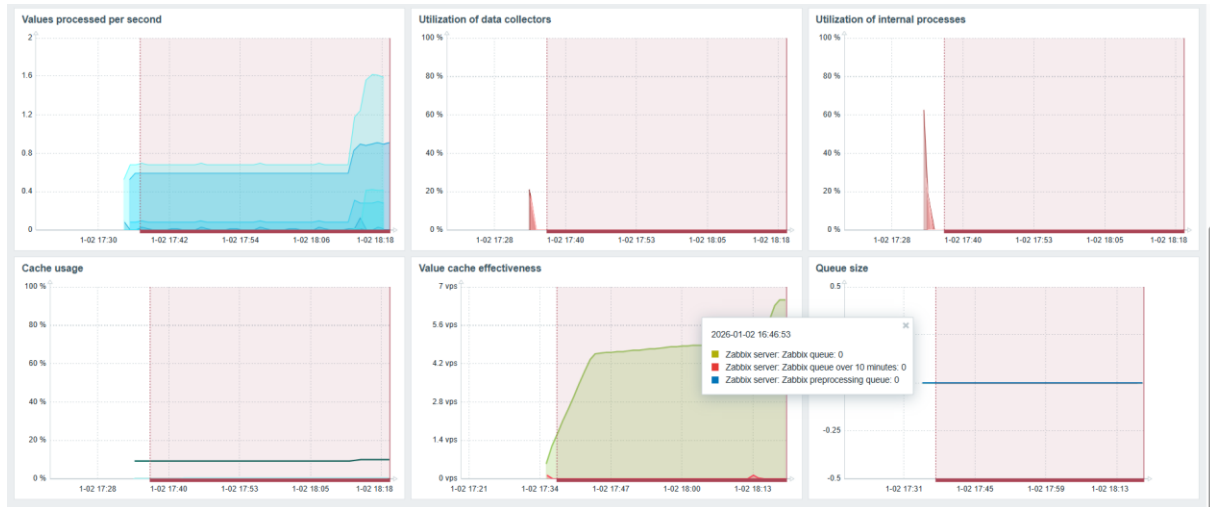


Figure 53: Graphique de charge CPU

Ces données sont affichées sous forme de **graphiques dynamiques** dans l'onglet **Monitoring** → **Latest Data**.

Non sécurisé 44.201.172.26/latest.php?ddreset=1

ZABBIX Monitoring Inventory Reports Configuration Administration

Dashboard Problems Overview Web **Latest data** Graphs Screens Maps Discovery Services

Latest data

Host groups: Linux servers (selected) Select

Hosts: Client-Linux (selected) Select

Application: Select

Apply Reset

Filter

Host Name Last check Last value Change

Specify some filter condition to see the values.

0 sélectionné Display stacked graph Display graph

Non sécurisé 44.201.172.26/latest.php?groupids%5B%5D=28&hostids%5B%5D=10318&application=&select=&show_without_data=1&filter_set=1

Host	Name	Last check	Last value	Change
Client-Linux	CPU (17 items)			
	Context switches per second	2026-01-02 18:26:00	76.9296	+5.281
	CPU guest nice time	2026-01-02 18:26:02	0 %	
	CPU guest time	2026-01-02 18:26:01	0 %	
	CPU idle time	2026-01-02 18:26:03	99.5665 %	+0.0666 %
	CPU interrupt time	2026-01-02 18:26:04	0 %	
	CPU iowait time	2026-01-02 18:26:05	0.0167 %	
	CPU nice time	2026-01-02 18:26:06	0 %	
	CPU softirq time	2026-01-02 18:26:07	0.0083 %	
	CPU steal time	2026-01-02 18:26:08	0.075 %	-0.025 %
	CPU system time	2026-01-02 18:26:09	0.1334 %	
	CPU user time	2026-01-02 18:26:10	0.2084 %	+0.0083 %
	CPU utilization	2026-01-02 18:26:03	0.4335 %	-0.0666 %
	Interrupts per second	2026-01-02 18:25:55	74.6306	-1.1148
	Load average (1m avg)	2026-01-02 18:25:57	0	
	Load average (5m avg)	2026-01-02 18:25:58	0	
	Load average (15m avg)	2026-01-02 18:25:56	0	
	Number of CPUs	2026-01-02 18:12:59	2	
Client-Linux	Disk nvme0n1 (6 items)			
	nvme0n1: Disk average queue size (avgqu-sz)	2026-01-02 18:26:23	0.0014	-0.0002
	nvme0n1: Disk read rate	2026-01-02 18:26:23	0 r/s	
	nvme0n1: Disk read request avg waiting time (r_await)	2026-01-02 18:25:56	0 ms	
	nvme0n1: Disk utilization	2026-01-02 18:26:23	0.025 %	-0.01 %

Listes des figures

Figure 1:Accès au Learner Lab AWS	8
Figure 2:Accès au Learner Lab AWS (Modules)	8
Figure 3:La connexion réussie du Lab.....	10
Figure 4:La sélection de la région us-east-1 (N. Virginia)	10
Figure 5:Création du VPC	11
Figure 6:Les paramètres principaux configurés lors de la création du VPC	12
Figure 7:L'activation des fonctionnalités DNS	12
Figure 8: Création du VPC dédié à l'infrastructure de supervision Zabbix	13
Figure 9:VPC	13
Figure 10:Etape 1- Passerelle Internet	14
Figure 11:Etape 2- Créer une Passerelle Internet	14
Figure 12: Création réussie de la passerelle Internet (Internet Gateway)	15
Figure 13:Créer une table de routage.....	15
Figure 14: Création réussie de la table de routage associée au sous-réseau public	15
Figure 15:Modification des routes de la table de routage.....	16
Figure 16:Succès de la modification de la table de routage.....	16
Figure 17:Créer un sous-réseau public	17
Figure 18:Les paramètres de configuration d'un sous- réseau	17
Figure 19:l'option Auto-assign public IPv4 address a été activée.....	19
Figure 20:Mappage des ressources	19
Figure 21:Sélectionner le service ECE	20
Figure 22:Cliquer sur groupe de sécurité.....	20
Figure 23:Créer un groupe de sécurité.....	21
Figure 24:Configurer les règles entrantes	22
Figure 25:Création des deux groupes de sécurité, Zabbix-Server-SG pour le serveur et Zabbix- Clients-SG.....	25
Figure 26:Lancer l'instance pour démarrer la création d'une nouvelle instance	27
Figure 27:Choix de l'AMI	28
Figure 28:Modification de réseau	28
Figure 29:Configuration des paramètres réseau	29
Figure 30:Clé SSH	29
Figure 31:Lancement de l'instance	30
Figure 32:Création réussie de l'instance EC2 du serveur Zabbix.....	30
Figure 33:Succès du lancement de l'instance Linux.....	31
Figure 34:Succès du lancement de l'instance Windows.....	32
Figure 35:Les trois instances EC2 déployées	33
Figure 36:Client SSH.....	35
Figure 37:Connexion à l'instance Douaa-Zabbix-Server via SSH	35
Figure 38:Mise à jour des paquets	36
Figure 39:Installation de Docker	36

Figure 40:Activation et démarrage du service Docker	37
Figure 41:Vérification de l'installation.....	37
Figure 42:Installation de Docker-Compose	38
Figure 43:Vérification de l'installation.....	38
Figure 44:Création du fichier docker-compose.yml	39
Figure 45:Lancement des conteneurs Zabbix	40
Figure 46:Vérification du fonctionnement.....	40
Figure 47:Interface Web de Zabbix affichée dans le navigateur	41
Figure 48:Tableau de bord Zabbix prêt à recevoir les clients.....	42
Figure 49:Ajout hôte Linux dans l'interface Zabbix	45
Figure 50:Statut "Vert" (ZBX) de client Linux	45
Figure 51:Configuration du fichier zabbix_agentd.conf.....	48
Figure 52:Statut "Vert" (ZBX) de client Windows.....	49
Figure 53:Graphique de charge CPU.....	50

Conclusion

Ce projet de mise en œuvre d'une **infrastructure cloud de supervision centralisée sous AWS** a été une expérience extrêmement enrichissante et formatrice. Il m'a permis de mettre en pratique les connaissances théoriques acquises au cours du semestre et d'approfondir mes compétences techniques dans plusieurs domaines clés.

Grâce à ce projet, j'ai pu :

- Déployer et configurer un **serveur Zabbix conteneurisé** via Docker, garantissant une supervision efficace des instances Linux et Windows.
- Installer et configurer les **agents Zabbix** sur les deux clients, permettant la collecte en temps réel des métriques système (CPU, RAM, disque, etc.).
- Concevoir une architecture cloud fonctionnelle sur AWS, incluant **VPC, sous-réseau public, passerelle Internet, tables de routage et groupes de sécurité**, en respectant les bonnes pratiques de sécurité et de connectivité.
- Vérifier le bon fonctionnement des services et visualiser les données dans **les tableaux de bord de Zabbix**, avec la génération de graphiques et la détection d'alertes.

Ce projet m'a également offert l'occasion de développer des compétences pratiques supplémentaires, notamment :

- L'utilisation de **Docker et Docker-Compose** pour orchestrer des conteneurs.
- La gestion et la configuration d'infrastructures cloud sur AWS.
- La compréhension des concepts de supervision d'un **parc hybride Linux/Windows**.

La principale difficulté rencontrée lors de ce projet a été de **voir l'interface Zabbix correctement et d'obtenir le statut "Vert" pour le client Windows**.

Grâce à la persévérance et à la vérification des paramètres réseau, des services Windows et de la configuration de l'agent, cette difficulté a été surmontée, et le client Windows est désormais correctement supervisé par le serveur Zabbix.

Enfin, cette expérience a renforcé ma capacité à **planifier, déployer et monitorer une infrastructure IT**, tout en respectant les contraintes techniques et sécuritaires. Elle constitue un atout précieux pour mon parcours académique et professionnel, et m'encourage à approfondir mes connaissances dans les domaines du cloud, de la virtualisation et de la supervision des systèmes.